

VIETNAMESE – GERMAN UNIVERSITY

**Faculty of Engineering
Computer Science Program**

**Design and Pilot Protocol of a Next.js Boilerplate
Framework for AI-Assisted SaaS Development**

Full name: Do Hoang Trieu

Student ID: 10422120

First supervisor: Dr. Le Lam Son

Second supervisor: Dr. Tran Huu Tam

BACHELOR THESIS

Submitted in partial fulfillment of the requirements for the degree of Bachelor Engineering
in study program Computer Science, Vietnamese - German University, 2026

Ho Chi Minh City, Viet Nam

Declaration

I, Do Hoang Trieu, hereby declare that this thesis, entitled “Design and Pilot Protocol of a Next.js Boilerplate Framework for AI-Assisted SaaS Development,” is my own original work. All sources of information have been properly acknowledged and referenced.

In preparing this thesis, I used AI tools to assist with code generation, prose drafting, and information research. I reviewed the output against the original sources and the SmartKit codebase, and take full responsibility for the content.

This thesis has not been submitted, in whole or in part, for the award of any other academic degree at this or any other institution. The author agrees that this thesis, upon acceptance, will be archived in the library of Vietnamese–German University and made available for reference use in accordance with the university’s regulations.

Acknowledgements

This thesis represents the capstone of my Bachelor's degree, a milestone that would not have been achieved without the invaluable support of many individuals. Therefore, I would like to express my deepest appreciation to those who have accompanied me throughout this journey.

My family has been the cornerstone of this achievement. Their unwavering encouragement, countless sacrifices, and steadfast belief in my abilities carried me through every stage of this work. Without their quiet strength and constant support, none of what follows would have been possible.

I am profoundly grateful to my supervisors for their dedicated mentorship. My first supervisor, **Dr. Le Lam Son**, provided exceptional guidance, boundless patience, and steady encouragement throughout this research. His rigorous academic oversight, incisive feedback, and continuous support were invaluable in sharpening the technical depth of this thesis. Beyond this project, I remain deeply grateful for his unwavering commitment, inspiring leadership, and the profoundly positive influence he continues to exert on our student community.

I am equally indebted to my second supervisor, **Dr. Tran Huu Tam**, whose mentorship was instrumental in transforming a vague idea into a coherent piece of engineering work. His dedication and encouragement helped shape this project from its earliest stages into a structured thesis.

I would also like to thank the **Faculty of Engineering** and the **Department of Computer Science** at the **Vietnamese–German University** for laying a solid academic foundation over the past four years and for fostering the environment in which this work could take shape.

Finally, my thanks go to my **university peers and friends**, whose companionship through the shared struggles and memorable moments of our undergraduate years made this journey not only bearable, but truly meaningful.

Abstract

AI coding assistants such as GitHub Copilot and Cursor AI are now in daily use, with substantial reported gains for boilerplate-heavy tasks: Peng et al. report a 55.8% completion-time reduction on a controlled HTTP-server task (Peng et al., 2023), and Yetistiren et al. find boilerplate generation the strongest use case among practitioners (Yetistiren et al., 2022). The benchmarks that dominate the literature, including HumanEval, MBPP+, APPS, Big-CodeBench, and LiveCodeBench for algorithms and SWE-bench for bug fixing, measure problem-solving and patch generation, not whether AI-generated code respects framework conventions. Server/Client boundary violations in Next.js App Router, for instance, fail at runtime in a way no static type check catches.

This thesis proposes a three-dimensional evaluation framework for AI-generated Next.js boilerplate and instantiates it in SmartKit, an open-source Next.js 16.2.12 boilerplate that acts as both the framework’s target codebase and a usable SaaS scaffold. The framework scores each generated feature on Structural Integrity, Type & Contract Safety, and Coverage & Completeness, each reported on a 0–5 scale. A pre-registered adaptive pilot ran 8 times on one feature: $n = 5$ in Condition A (SmartKit) after a mid-study extension, $n = 3$ in Condition B (convention-naïve scaffold). Two of three directional predictions are confirmed outright; the third is asymmetric, confirmed in Condition A and refuted in Condition B, and is reported as such rather than collapsed into a single verdict.

The pilot is feasibility-stage evidence rather than an inferential test. Per-arm samples are too small for inferential statistics, and three protocol adaptations are disclosed: an adaptive extension applied to Condition A only, a tool substitution for runs 6–8, and two-clause reporting of the asymmetric prediction introduced after data collection. Larger-N replication, cross-tool validation, and developer-participant studies remain future work.

Contents

Declaration	i
Acknowledgements	ii
Abstract	iii
1 Introduction and Motivation	1
1.1 Overview	1
1.2 The Problem: AI Convention Violations in Next.js	2
1.3 Research Objectives and Questions	2
1.4 Scope and Limitations	6
2 Background	8
2.1 AI Coding Assistants	8
2.2 Next.js Framework Architecture	8
2.3 SaaS Boilerplate Patterns	9
3 Related Work	10
3.1 Code Generation Benchmarks	10
3.2 Empirical Studies on Developer Productivity	11
3.3 Evaluation Methodology	12
3.4 Research Gap Summary	12
3.5 Literature Search Methodology	13
3.6 Alternative Approaches to AI-Codebase Alignment	13
4 Proposed Solution	16
4.1 Design Rationale	16
4.2 Feature-Based Modular Architecture	16
4.3 Server/Client Boundary Conventions	17
4.4 Database Schema Patterns	17
4.5 Type-Safe API Contracts	18
5 Implementation	19
5.1 Technology Stack	19
5.2 Project Structure	19
5.3 Key Implementation Patterns	20
5.4 Billing and SePay Integration	21
6 Evaluation	22
6.1 Evaluation Framework	22
6.2 Design Rationale and Trade-offs	24
6.3 Evaluation Scenarios	25
6.4 Control Comparison	25
6.4.1 Design Rationale	26
6.4.2 Experiment Protocol	27

6.4.3	Expected Threats and Mitigations	28
6.5	Multi-Run Evaluation	28
6.5.1	Non-Determinism Threat	28
6.5.2	Experimental Protocol	28
6.5.3	Statistical Reporting	29
6.5.4	Implications for Framework Reliability	29
6.6	Evaluation Metrics	30
6.7	Pilot Evaluation Protocol	30
7	Conclusion and Future Work	35
7.1	Summary	35
7.2	Limitations and Threats to Validity	35
7.3	Diffusion Pathway and Adoption Risks	37
7.4	Future Work	38
A	Pilot Pre-Registration — SmartKit C2	40
A.1	Pilot Summary	40
A.2	Pilot Feature	40
A.3	Conditions	41
A.3.1	Condition A (SmartKit)	41
A.3.2	Condition B (baseline)	41
A.3.3	Run Schedule (as actually executed)	42
A.3.4	Binary Verdict Rule (current)	42
A.3.5	Inter-Rater Reliability	42
A.4	Prompt Template	42
A.5	Scoring Rubric	43
A.5.1	Dimension 1: Structural Integrity (0–5)	44
A.5.2	Dimension 2: Type & Contract Safety (0–5)	44
A.5.3	Dimension 3: Coverage & Completeness (0–5)	45
A.5.4	Aggregate Score	45
A.6	Exclusion Criteria	46
A.7	Lock and Amendment Procedure	46
A.8	Snapshot Limitation	47
A.9	References	47
A.10	Pilot Run Log	48
A.11	Amendment Log	49
A.12	JSON Output Schema	51

List of Figures

1.1 AI-generated Next.js App Router code violates Server/Client boundaries by passing event handlers across component types, producing runtime errors that static analysis cannot detect. 5

1.2 Three orthogonal evaluation dimensions (structural integrity, type & contract safety, coverage & completeness) applied to a target corpus of $5 \times 3 \times 10 = 150$ AI-generated artifacts; the executed pilot exercises $1 \times 3 \times 8 = 8$ runs. . . . 6

List of Tables

3.1	Qualitative positioning of SmartKit against established Next.js starter templates. Cells describe documented features, not measured performance; cell-level claims were transcribed from each template’s own documentation.	14
6.1	Reference coverage checklist for the <code>isEnabled</code> feature flag utility.	24
6.2	Condition A (SmartKit at full setup) vs. Condition B (convention-naïve scaffold) used in the pilot evaluation.	26
6.3	Pilot scoring output for all 8 runs: Condition A ($N = 5$, Runs 4–8) vs. Condition B ($N = 3$, Runs 1–3); Str / T&C / Cov on the 0–5 rubric.	32
6.4	Per-run automated vs. manual score cross-check across all eight pilot runs; maximum per-dimension absolute delta = 1 across 24 (run, dimension) pairs.	33
A.1	Pilot summary	40
A.2	Pilot run schedule as executed ($N_A = 5$, $N_B = 3$); see §A.11 for amendment details.	42
A.3	Structural Integrity scoring rubric (0–5)	44
A.4	Type Safety sub-score rubric (0–5)	44
A.5	Contract Safety sub-score rubric (0–5)	45
A.6	Coverage & Completeness scoring rubric (0–5)	45
A.7	Exclusion criteria for pilot runs	46
A.8	Pilot run log (8 runs): per-run Str / Type / Cov / Tot scores and DP1–DP3 verdicts; Wilson 95% CIs in §6.7.	48
A.9	Amendment log (locked + 10 amendments). Source: docs/pilot-pre-registration.md §11.	50
A.10	JSON output schema for pilot runs (matches SmartKit/scripts/score-flag.ts v1.2).	52

1. Introduction and Motivation

The chapter opens with the recent wave of AI coding assistants and the parallel shift in Next.js toward the App Router, then identifies the absence of an evaluation framework for framework-specific convention adherence. From that gap the chapter derives the research objectives, questions, and hypotheses that structure the rest of the thesis, and closes with the scope within which the SmartKit framework is designed and evaluated.

1.1 Overview

Controlled studies report substantial productivity gains from AI coding assistants. Peng et al. showed that developers using GitHub Copilot completed tasks 55.8% faster than control groups (Peng et al., 2023), while Yetistiren et al. found AI tools particularly effective for boilerplate-generation tasks (Yetistiren et al., 2022). Next.js has emerged as the dominant React meta-framework for production web applications, with its App Router architecture introduced in version 13 fundamentally changing application structure through React Server Components and explicit Server/Client boundaries (Vercel, 2024). The failure mode that motivates this thesis is concrete (Figure 1.1): when an AI coding assistant generates a Server Component that imports a Client Component and passes an event handler as a prop, Next.js produces the runtime error “Event handlers cannot be passed to Client Component props”—a failure that `tsc --noEmit` cannot catch because it has no type-level representation. During SmartKit’s development, recurring AI generation errors were observed firsthand in Next.js App Router code, including authentication security flaws, component boundary violations, and outdated API usage from older training data; these are development-time observations rather than measured prevalence figures. Without systematic evaluation, developers cannot assess whether AI-generated Next.js code adheres to framework conventions or will fail in production.

Existing benchmarks address algorithmic problem-solving and bug-fixing capabilities but not framework convention adherence. HumanEval evaluates Python code generation for standalone functions (Chen et al., 2021); SWE-bench assesses AI ability to resolve GitHub issues (Jimenez et al., 2024); RigorBench (a recent, as-yet-unreviewed preprint, with a self-disclosed author conflict of interest) measures process-discipline metrics (Madiraju & Madiraju, 2026). None of them capture the specific architectural requirements of Next.js App Router, such as Server/Client component boundaries, type-safe contracts, or file-system routing conventions. The gap means developers have no quantitative basis for comparing AI tools on framework-specific tasks.

Intended users. SmartKit targets solo developers and small teams (2–5 members) working on Next.js projects who use AI coding assistants (primarily Cursor AI and GitHub Copilot) for boilerplate generation. The intended users are intermediate developers who understand React and TypeScript fundamentals but seek to reduce boilerplate setup time and enforce consistent conventions in AI-assisted development. The framework supports incremental adoption: teams can apply the feature-based folder structure and Server/Client boundary conventions without committing to the full stack.

1.2 The Problem: AI Convention Violations in Next.js

This thesis addresses a concrete problem: AI coding assistants consistently generate Next.js App Router code that violates framework conventions, producing runtime errors and technical debt, yet no evaluation framework exists to measure this adherence. Designing such an evaluation framework faces three principal challenges. First, framework conventions are implicit and distributed across documentation, error messages, and community patterns, making them difficult to enumerate into testable metrics. Second, AI-generated code failures manifest at runtime rather than compile time. Server/Client boundary violations produce “Event handlers cannot be passed to Client Component props” errors only when executed, so they require dynamic analysis rather than static linting alone. Third, the evaluation dimensions must be designed to be orthogonal: structural integrity violations do not imply type errors, and coverage gaps operate independently of both, demanding a multi-dimensional framework rather than a single composite score.¹

The focus on Next.js App Router is deliberate. First, market dominance makes it the primary target: community surveys consistently place Next.js among the most widely adopted React meta-frameworks (Vercel, 2024); Stack Overflow’s annual Developer Survey has tracked Next.js as the most-used React meta-framework since 2021 (Stack Overflow, 2025, Stack Overflow Developer Survey). Second, training-data mismatch creates measurable failures: AI tools generate outdated patterns from older training data that conflict with current Next.js conventions, particularly around Server/Client boundaries and async API changes. Third, practical feasibility applies: the author has existing Next.js expertise, enabling direct testing of enforcement mechanisms. The pilot targets Cursor AI as the primary case study because its AGENTS.md feature implements documentation-based guidance, which prior research on grounded prompting suggests can improve AI output quality (Barke et al., 2022; Zhang et al., 2023).

1.3 Research Objectives and Questions

While Next.js dominates React meta-framework adoption and AI coding assistants such as GitHub Copilot and Cursor AI have demonstrated substantial productivity gains, there is a distinct gap at the intersection of these two areas: a systematic, operationalizable evaluation framework that measures how closely AI-generated Next.js boilerplate code adheres to framework-specific conventions. Existing benchmarks—HumanEval, SWE-bench, RigorBench—address algorithmic problem-solving and bug-fixing capability, but none capture the architectural requirements of Next.js App Router such as Server/Client component boundaries, type-safe contracts, or file-system routing conventions (Chen et al., 2021; Jimenez et al., 2024; Madiraju & Madiraju, 2026). This thesis targets precisely that gap.

The primary motivation of this thesis is to bridge this gap by designing a three-dimensional evaluation framework that turns an implicit set of framework conventions into measurable, comparable scores, and by instantiating that framework in SmartKit, a structured Next.js boilerplate that provides the necessary enforcement mechanisms and the evaluation target codebase.

The research pursues three objectives:

¹The orthogonality of the three dimensions is asserted at design stage and awaits empirical validation; the pilot’s per-arm sample is far below the $n \approx 30$ required to confirm that the dimensions vary independently in practice. See §7.4 for the sample-size discussion that bounds this claim.

- **Design a three-dimensional evaluation framework.** Construct a framework that scores AI-generated Next.js boilerplate code along three dimensions: Structural Integrity, Type & Contract Safety, and Coverage & Completeness, so that convention adherence becomes a quantitative, comparable property of generated code.
- **Implement SmartKit as a structured architecture.** The thesis releases SmartKit, an open-source Next.js boilerplate that demonstrates feature-based modularity, explicit Server/Client boundaries, and type-safe contracts, providing both the enforcement mechanisms and the target codebase for the framework.
- **Conduct a pilot empirical case study.** The thesis applies the framework to a representative feature generated by Cursor AI under the SmartKit condition to demonstrate that the framework can produce illustrative, actionable scores on real generated code.

Consequently, this research seeks to answer the following research questions:

1. How can an evaluation framework be consistently designed to measure AI coding assistant adherence to Next.js App Router boilerplate conventions along the Structural Integrity, Type & Contract Safety, and Coverage & Completeness dimensions?
2. Can the resulting framework be operationalized into measurable metrics, and does it produce illustrative, actionable scores when applied to representative AI-generated Next.js features?

These objectives and questions give rise to two classes of claims, both documented as design-level pre-registrations to be exercised by the pilot (Section 6.7). The first class states **framework properties**—descriptive claims about whether the framework can measure the dimensions it targets. The second class states **directional predictions**—falsifiable claims about the sign of the score difference between conditions, to be reported as confirmed, refuted, or inconclusive per Section 6.7.

Framework properties. The pilot is designed to demonstrate that the framework can be operationalised on a real artifact:

- **FP1 (Structural Integrity).** The framework can measure file placement, Server/Client boundary usage, and naming patterns in AI-generated Next.js code, producing a 0–5 score per run on a representative feature.
- **FP2 (Type & Contract Safety).** The framework can measure TypeScript error counts and Zod-validation coverage in AI-generated Next.js code, producing a 0–5 score per run that combines a Type-Safety sub-score and a Contract-Safety sub-score via the weakest-link rule.
- **FP3 (Coverage & Completeness).** The framework can enumerate AI-generated files against a feature-specific checklist and produce a 0–5 per-run score that reflects required-artifact presence.

Directional predictions. Each directional prediction states the expected sign of the difference between Condition A (SmartKit at full setup) and Condition B (convention-naïve scaffold) on a representative feature—the `isEnabled(flagKey, userId?)` utility, which uses no service that SmartKit integrates. Predictions are evaluated as *confirmed* (all runs in the predicted direction), *refuted* (no runs in the predicted direction), or *inconclusive* (intermediate), per the adaptive-N protocol documented in Section 6.7.

- **DP1 (Structural).** Condition A is predicted to place the utility inside `features/flags/lib/is-enabled.ts` in the majority of runs; Condition B is predicted to place it at `lib/feature-flags.ts` or at the repository root.
- **DP2 (Type & Contract).** Condition A is predicted to export a Zod-derived `FlagKey` enum or branded type in the majority of runs; Condition B is predicted to use raw string keys.
- **DP3 (Coverage).** Condition A is predicted to include a centralized `src/tests/features/flags/is-enabled.test.ts` in the majority of runs; Condition B is predicted to include no test file in the majority of runs.

Framework properties and directional predictions serve different purposes. A framework property is a descriptive claim about the framework itself; it cannot be refuted by pilot data, only supported or unsupported. A directional prediction is a falsifiable claim about the artifact and is the only class of claim to which the binary *confirmed/refuted/inconclusive* verdict applies. The pilot protocol (Section 6.7) documents how each is reported. Full statistical validation—including larger sample sizes, cross-tool replication, and developer-participant studies—remains future empirical work.

Designing an evaluation framework for AI-generated Next.js boilerplate encounters three key challenges that distinguish it from existing code generation benchmarks. First, framework conventions are implicit and distributed across documentation, error messages, and community patterns, requiring deliberate extraction into enumerable, testable rules rather than relying on predefined quality criteria. Second, convention violations in Next.js App Router manifest primarily at runtime. Server/Client boundary violations produce errors only when executed, so evaluation must go beyond static analysis to capture dynamic execution behaviour. Third, the evaluation dimensions are orthogonal: structural integrity violations do not imply type errors, and coverage gaps operate independently of both, demanding a multi-dimensional framework rather than a single composite score.²

The contributions of this thesis are as follows. First, it proposes a three-dimensional evaluation framework (Structural Integrity, Type & Contract Safety, and Coverage & Completeness) for measuring how closely AI coding assistants adhere to Next.js App Router conventions. The framework addresses a gap left by existing benchmarks, which score algorithmic correctness rather than framework-specific quality. Second, it releases SmartKit, an open-source Next.js boilerplate demonstrating feature-based modularity, explicit Server/Client boundaries, and type-safe contracts. SmartKit supplies both the conventions the framework measures and the target codebase it measures them on; all five Better Auth tables (user, session, account, verification), the billing feature (SePay integration with HMAC signature verification and idempotent webhook handling), and the email integration (Resend + React Email) are implemented at the current snapshot rather than deferred to future phases, supporting SmartKit's role as a functional SaaS boilerplate. Third, it releases the pilot protocol and the baseline comparison design as artefacts: a pre-registration document, a Condition A vs. Condition B comparison scaffold, a scoring rulebook, and a baseline-bootstrap script. The artefacts are concrete enough that another researcher can replicate the pilot without re-deriving the protocol from the thesis prose.

²As in §1.3 above, this orthogonality claim is asserted at design stage and awaits empirical validation; see §7.4 for the sample-size discussion that bounds it and the planned Pearson correlation analysis that would distinguish genuine orthogonality from subtle coupling.

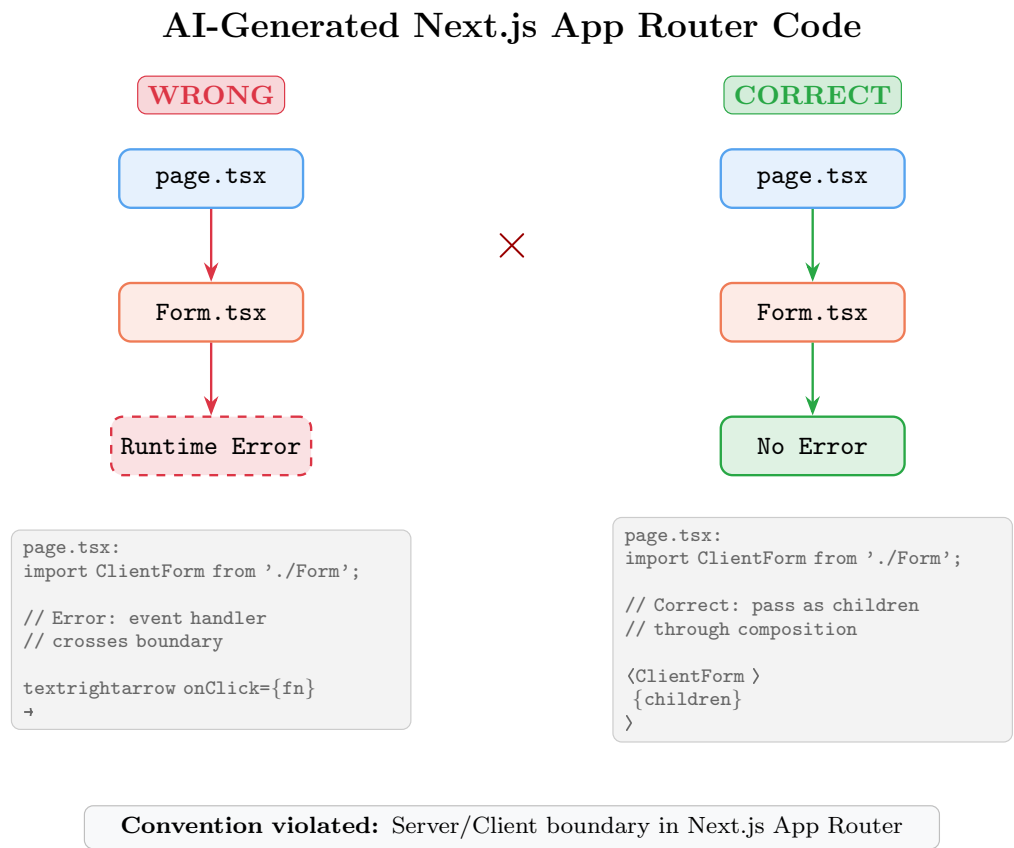


Figure 1.1: AI-generated Next.js App Router code violates Server/Client boundaries by passing event handlers across component types, producing runtime errors that static analysis cannot detect.

Figure 1.2 summarizes the resulting evaluation-framework architecture. Each of the three dimensions is paired with an automated measurement: Structural Integrity is measured through parsed-AST rule violations, Type & Contract Safety through TypeScript compiler output, and Coverage through a rule-coverage matrix checked against the convention catalog. The three measurements are applied to a shared corpus of AI-generated artifacts and produce a per-tool violation matrix, type-error rates, and coverage scores. Because the dimensions are deliberately independent, no single composite score can substitute for examining all three independently.

Evaluation Framework Architecture

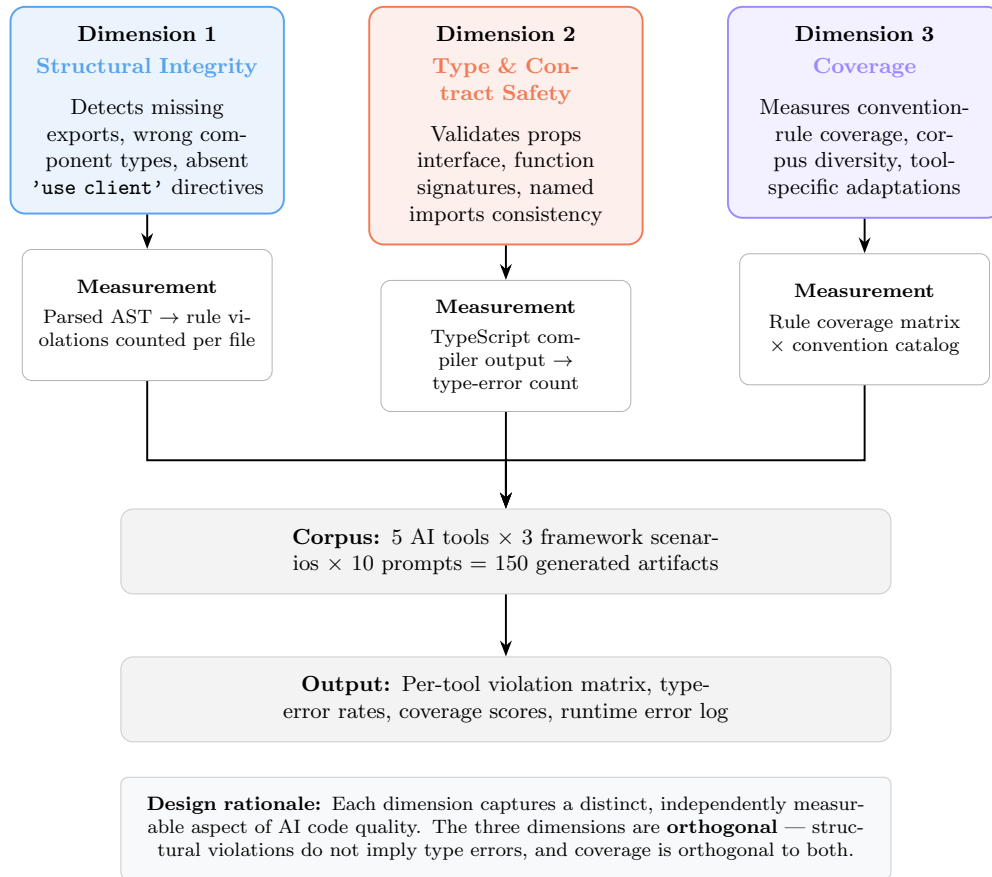


Figure 1.2: Three orthogonal evaluation dimensions (structural integrity, type & contract safety, coverage & completeness) applied to a target corpus of $5 \times 3 \times 10 = 150$ AI-generated artifacts; the executed pilot exercises $1 \times 3 \times 8 = 8$ runs.

This corpus scale represents the target scope for full empirical validation (Section 7.4); the thesis pilot (Section 6.7) applies the framework to a single representative feature within this scope.

1.4 Scope and Limitations

To keep the empirical rigor and feasibility of this research within the resource envelope of a single-author thesis, the scope is strictly defined by the following parameters and limitations:

1. **Framework scope:** Next.js App Router only. No comparison with Nuxt, Remix, or other meta-frameworks.
2. **AI assistants:** Cursor AI used for the pilot empirical case study. Broader AI assistant evaluation (GitHub Copilot, Claude Code) is future work.
3. **Project type:** Single representative Next.js project demonstrating SaaS patterns.

4. **Pilot study limitation:** The empirical case study is a worked-example executed against a total of 8 runs under a pre-registered adaptive schedule, distributed as $n = 5$ Cond A and $n = 3$ Cond B on a single representative feature, the `isEnabled(flagKey, userId?)` utility, which uses no service that SmartKit integrates. The pilot illustrates framework applicability and produces per-condition medians plus directional predictions (DP1–DP3, Section 6.7); two of three are confirmed outright and the third is split between conditions. Cond B remains at $n = 3$, so the per-arm sample is still far below the $n \approx 30$ required for inferential validation (Section 7.4). Cross-tool replication, larger- N studies, and developer-participant validation remain future empirical work.

These boundaries are deliberate methodological choices. Fixing the framework, the AI assistant, and the project type removes confounding variance so that any measured difference between structured and unstructured conditions can be attributed to the SmartKit approach alone.

2. Background

Understanding the intersection of AI coding assistants and Next.js development requires grounding in three domains: the capabilities and limitations of current AI coding tools, the architectural patterns of the Next.js framework, and the principles of SaaS boilerplate design.

2.1 AI Coding Assistants

GitHub Copilot, Anthropic’s Claude, and Cursor AI all use transformer-based LLMs fine-tuned for code understanding and generation. GitHub Copilot provides inline code completions and entire function generation within supported editors (using OpenAI’s GPT series and Anthropic’s Claude series models, with specific model versions varying over time). Anthropic’s Claude offers conversational code assistance through the Claude Code CLI and IDE integrations. Cursor combines Claude with a custom codebase indexing system that enables context-aware suggestions across entire projects.

These assistants share common architectural elements. They process code through transformer-based models trained on large corpora of publicly available source code, learning statistical patterns that inform code generation. When producing completions, they consider surrounding code context, natural language comments, and project-wide information to generate relevant suggestions. Their effectiveness varies with programming language, task complexity, and code structure (Yetiştiren et al., 2023).

Research documents both strengths and limitations. Dakhel et al. found that AI assistance benefits expert developers who can filter suboptimal suggestions but may harm novice developers who lack the expertise to identify incorrect or inefficient code (Dakhel et al., 2023). Liu et al. documented that ChatGPT performs better on older problems (pre-2021) than newer ones, with a 48.14% advantage on the Accepted rate, though it struggles with complex algorithms and domain-specific code, frequently generating solutions with security vulnerabilities and inefficient implementations (Z. Liu et al., 2024).

The task taxonomy for AI suitability shows that boilerplate generation represents a strength area. Zhang et al. observed that developers primarily use Copilot for boilerplate generation, test writing, and code explanation rather than novel algorithmic development (Zhang et al., 2023). Pandey et al. confirmed this pattern, finding strong AI performance on template generation and documentation tasks but weak performance on complex logic and integration code (Pandey et al., 2024). These findings focus the study on boilerplate interaction as an appropriate domain for AI enhancement.

2.2 Next.js Framework Architecture

Next.js is a React meta-framework that provides server-side rendering, static site generation, and API capabilities within a unified application model. Version 13 introduced the App Router paradigm, fundamentally changing how React applications structure pages, layouts, and data fetching. Developers who learned the Pages Router encounter several App Router concepts that the older model did not surface, including Server/Client boundaries and file-system conventions for special files.

React Server Components represent a core innovation in Next.js App Router. These components execute exclusively on the server, which lets them access databases and sensitive operations directly without exposing logic to the client. The distinction between Server and Client components requires explicit marking through the `'use client'` directive, creating a boundary that developers must carefully manage. Incorrect placement of components across this boundary produces runtime errors and performance degradation.

The App Router organizes applications through a file-system-based routing mechanism. Each folder within the `app/` directory corresponds to a route segment, with `page.tsx` files defining the content for each route. Special files such as `layout.tsx`, `loading.tsx`, and `error.tsx` provide shared UI and error handling across route hierarchies. This convention-driven structure enables rapid scaffolding but requires adherence to specific naming and placement patterns.

TypeScript integration in Next.js emphasizes type safety throughout the application. The framework provides typed interfaces for page properties, API request and response objects, and data fetching results. Achieving full type coverage requires consistent use of TypeScript patterns including generics, utility types, and strict mode configuration. AI coding assistants may generate code that compiles but violates framework conventions, producing runtime errors or subtle bugs.

2.3 SaaS Boilerplate Patterns

Software-as-a-service applications share common requirements that boilerplate frameworks address: user authentication, database access patterns, multi-tenancy support, and deployment configurations. The economics of SaaS development favor approaches that reduce per-project initialization effort while maintaining code quality and consistency.

Krüger et al. established that platform reuse reduces maintenance effort but involves upfront investment in framework adoption and learning curve (Krüger & Berger, 2020). The trade-off explains the appeal of well-designed boilerplate templates that amortize initialization cost across multiple projects. Aleem et al. identified customization, scalability, multi-tenancy, security, integration, and fault tolerance as key factors in SaaS architecture decisions (Aleem et al., 2019).

Modern scaffolding approaches build on model-driven architecture principles. Inayatullah and Azam showed that automated code generation reduces boilerplate development effort for cross-platform applications (Inayatullah et al., 2019). Liu and Sra recently showed that LLMs have potential for automatically generating scaffolded user interfaces, reducing complexity for users in professional creative software (Y. Liu & Sra, 2025).

Meta-framework evaluation identifies architectural trade-offs across options. Prior work on framework adoption in software engineering has identified learnability, extensibility, and ease of migration as primary drivers of adoption, while framework lock-in and high configurability are commonly cited as major adoption risks (Storey et al., 2016). Next.js App Router emphasizes React Server Components as a first-class primitive, while maintaining full compatibility with Client Components for interactive UI. The hybrid model creates specific conventions that boilerplate frameworks should encode.

3. Related Work

This section surveys existing research on code generation benchmarks, empirical studies of AI-assisted development, and systematic literature reviews on AI for software engineering. The analysis identifies the specific gap this thesis addresses: the absence of framework-specific evaluation for AI coding assistants operating on Next.js boilerplate conventions.

3.1 Code Generation Benchmarks

The evaluation of AI coding assistants relies heavily on standardized benchmarks that measure code generation capability. Chen et al. introduced HumanEval as a foundational benchmark containing 164 hand-written programming problems with function signatures, docstrings, and unit tests (Chen et al., 2021). The benchmark evaluates models using Pass@1, Pass@10, and Pass@100 metrics. OpenAI’s Codex achieved 28.7% Pass@1 on this benchmark, establishing a baseline for subsequent model development.

HumanEval focuses exclusively on single-function Python code generation with algorithmic and reasoning requirements. The design prioritizes general programming capability over domain-specific or framework-aware generation. Yu et al. extended the benchmark concept with HumanEval Pro, adding self-invoking code generation tasks that require models to generate functions calling previously defined functions (Yu et al., 2025). Their work documented up to 30% performance degradation on self-invoking tasks compared to standalone tasks, indicating that current benchmarks underestimate real-world code generation complexity.

SWE-bench represents a substantial advance in benchmark realism by evaluating AI models on real GitHub issues from popular Python repositories (Jimenez et al., 2024). Each task requires understanding the issue description, locating relevant code, implementing a fix, and passing unit tests. Jimenez et al. found that state-of-the-art models resolved only a small fraction of issues—Claude 2 solved approximately 1.96% and GPT-4 only 1.7%—and the gap between benchmark performance and practical software engineering capability is substantial.

Language-specific benchmarks identify further evaluation limitations. Petrukha et al. developed SwiftEval for Swift programming, finding that general-purpose benchmarks overestimated Swift code generation capability (Petrukha et al., 2025). Their language-specific benchmark documented 40–60% performance gaps not captured by cross-language evaluations.

No existing benchmark evaluates framework convention adherence, boilerplate structure quality, TypeScript or Next.js-specific patterns, or multi-file scaffolding tasks. The benchmarks measure whether AI assistants can solve algorithmic problems or fix bugs in existing codebases, not whether they can generate framework-compliant boilerplate from scratch.

Several extensions and alternatives warrant mention. Liu et al. developed HumanEval+ and MBPP+ as rigorous extensions of the original benchmarks, finding that pass rates drop often by 30% or more when evaluated against additional high-quality tests per problem (J. Liu et al., 2024). Zhuo et al. introduced BigCodeBench, which evaluates 1,140 fine-grained tasks combining diverse function calls with complex natural-language instructions (Zhuo et al., 2024). Hendrycks et al. proposed APPS, a 10,000-problem benchmark spanning introductory, interview, and competition-level coding challenges (Hendrycks et al., 2021). Jain et al. addressed the critical contamination problem through LiveCodeBench, which sources

problems from recent competitive programming contests published after model training cut-offs (Jain et al., 2024). Li et al. developed CodeReviewer for code review automation, an adjacent evaluation direction that assesses code quality dimensions relevant to AI-assisted development workflows (Li et al., 2022). Each of these benchmarks measures distinct aspects of code generation or code quality capability, yet none evaluate the framework-specific convention adherence that the SmartKit framework targets.

3.2 Empirical Studies on Developer Productivity

Several empirical studies examine how AI assistance affects real developer workflows. Peng et al. conducted the seminal controlled experiment on GitHub Copilot with 95 recruited developers implementing an HTTP server in JavaScript (Peng et al., 2023). Their methodology used a treatment-control design with matched tasks, finding the 55.8% completion time improvement cited earlier. Heterogeneous effects showed that less experienced developers benefited most from AI assistance.

The enterprise-level .NET study by Tariq et al. extended this methodology to enterprise software development scenarios (Tariq et al., 2026). With 60 developers and 960 task observations across eight enterprise-oriented tasks, they measured task completion time, implementation errors, cyclomatic complexity, maintainability index, code smells, and NASA-TLX cognitive load scores. AI-assisted development reduced task completion time, errors, code smells, and cognitive workload while improving maintainability. The trade-off was that AI generated redundant abstractions and increased code verbosity, with effectiveness depending on developer expertise and continued oversight.

Liu et al. assessed ChatGPT code generation quality across multiple languages and task types (Z. Liu et al., 2024). ChatGPT performs better on pre-2021 problems with a 48.14% advantage on Accepted rate, but its ability to directly fix erroneous code through multi-round dialogue is relatively weak. Generated code often contained security vulnerabilities and inefficient implementations requiring human verification, though ChatGPT could resolve over 89% of identified vulnerabilities through multi-round fixing. Yetiştiren et al. documented that Copilot generates functionally correct code for many problems but frequently produces solutions with higher cyclomatic complexity and lower readability than human-written solutions (Yetiştiren et al., 2022).

AlOmar et al. examined developer-ChatGPT refactoring conversations, finding that developers frequently needed multiple iterations to achieve satisfactory refactoring (AlOmar et al., 2024). AI often missed design-level improvements, and common anti-patterns emerged in developer-AI refactoring interactions. AI effectiveness therefore depends on the structure of the interaction and on the developer’s ability to guide refinement, not only on the tool itself.

Hou et al. conducted a systematic literature review of 395 research articles on LLMs for software engineering from 2017–2024 (Hou et al., 2024). Code generation (35%) and code repair (25%) are the most studied application areas. Most studies use benchmark evaluation (60%) rather than human evaluation, and reported effectiveness varies widely by task type. The review identified research gaps in security, requirements engineering, and empirical studies. Framework-specific boilerplate evaluation would begin to address these gaps.

Russo et al. analyzed GenAI adoption complexity, identifying five key dimensions: technical, organizational, individual, process-related, and ethical (Russo et al., 2024). Technical capability was necessary but insufficient; human factors including trust, skill, and workflow integration were critical success factors. Evaluating framework-specific boilerplate tools

therefore requires attention to how well they integrate with developer workflows and support skill development.

3.3 Evaluation Methodology

The refinement of AI-generated code has received attention as a complementary evaluation dimension. Liu et al. analyzed 4,066 ChatGPT-generated programs across 2,033 programming tasks in Java and Python (Y. Liu et al., 2024). Of these programs, 2,756 were deemed correct, 1,082 produced wrong outputs, and 177 contained compilation or runtime errors; 1,930 additional programs suffered from maintainability issues identified through static analysis. Their automated refinement pipeline using static-analysis feedback improved code quality by more than 20%. Evaluation frameworks should therefore measure both initial generation quality and the effort required for refinement.

Ouyang et al. studied the non-determinism of ChatGPT in code generation (Ouyang et al., 2025). Across 829 problems in three benchmarks, the ratio of coding tasks with zero equal test output across different requests ranged from 47.56% (HumanEval) to 75.76% (CodeContests). Setting the temperature to 0 does not guarantee determinism, despite being a common practice. Reliable assessment therefore requires multiple runs with confidence intervals rather than single-point measurements.

Beller et al. proposed the DAT framework for measuring software development productivity at Meta (Beller et al., 2026). The approach combines diff authoring time, survey-based metrics, and AI tool-specific productivity measures. The DAT framework models a path for evaluating AI-boilerplate interaction that balances quantitative performance metrics with qualitative developer experience assessment.

Nguyen-Duc et al. outlined a research agenda for GenAI in software engineering, emphasizing code generation quality, testing and verification, developer experience, education and training, ethics and security, and process and project management (Nguyen-Duc et al., 2025). The agenda explicitly calls for standardized evaluation and longitudinal studies, requirements that framework-specific evaluation frameworks can begin to satisfy.

Sauvola et al. projected four primary future scenarios for generative AI in software development, ranging from legacy integration to clean-slate and networked development, with software production shifting toward higher-level design and oversight roles as routine coding tasks become increasingly automated (Sauvola et al., 2024). Research on optimizing AI interaction patterns, including framework-specific boilerplate conventions that reduce the cognitive overhead of AI-assisted development, follows from this projection.

3.4 Research Gap Summary

The reviewed literature establishes that AI coding assistants improve developer productivity on repetitive tasks, that boilerplate generation represents an AI strength area, and that existing benchmarks measure algorithmic capability rather than framework convention adherence. To the author’s knowledge—given the structured-but-non-exhaustive literature search described in the next section—no prior work has specifically evaluated AI tool effectiveness on Next.js App Router conventions, React Server Components patterns, TypeScript-first boilerplate structures, or full-stack framework-specific best practices. This gap is independently supported by Hou et al. (2024)’s systematic review of 395 LLM-SE articles, which does not list framework-specific convention adherence as a separately tracked evaluation

category (Hou et al., 2024); the review’s documented research gaps (security, requirements engineering, empirical studies) likewise do not overlap with framework-specific boilerplate evaluation. Together with RigorBench’s complementary observation that existing benchmarks measure outcomes rather than process discipline (Madiraju & Madiraju, 2026), this triangulates the gap claim against a peer-reviewed systematic review rather than relying on a single, COI-disclosed preprint. This thesis proposes a Next.js boilerplate framework engineered for AI interaction and an evaluation methodology that measures framework-specific quality dimensions, reporting both as design-stage contributions.

3.5 Literature Search Methodology

The literature reviewed in this chapter was identified through a structured search process. Primary databases searched include ACM Digital Library, IEEE Xplore, arXiv, and Google Scholar. Search terms combined AI coding assistant terminology (“LLM code generation,” “Copilot evaluation,” “ChatGPT code quality”) with framework-specific terminology (“React Server Components,” “Next.js,” “TypeScript boilerplate”) and benchmark terminology (“code generation benchmark,” “framework-specific evaluation”). Inclusion criteria prioritized peer-reviewed publications from 2020 onward, with seminal pre-2020 works (e.g., HumanEval, Codex) included for foundational context. The 35 references in the bibliography are supplemented by the systematic literature review of Hou et al. (Hou et al., 2024) which provides independent coverage of 395 articles in the LLM-for-SE space. The author acknowledges that a fully PRISMA-compliant systematic review with explicit screening protocol is beyond single-author-thesis scope; the search described here is structured but not exhaustive, and additional framework-specific evaluation work may exist outside the search scope.

3.6 Alternative Approaches to AI-Codebase Alignment

The structural approach taken by SmartKit is one of several strategies that researchers and practitioners have explored to improve AI-codebase alignment. This section briefly reviews these alternatives and justifies why the structural approach was selected for this thesis.

Prompting strategies represent the most lightweight alternative. Enhanced system prompts, few-shot examples, and retrieval-augmented generation (RAG) can improve AI output quality without modifying project structure. Prompting approaches suffer from context window limitations, require repeated re-instruction across sessions, and cannot enforce consistency across a codebase without external governance. RigorBench (2026) found that existing benchmarks evaluate outcomes but not the engineering process through which AI agents arrive at solutions (Madiraju & Madiraju, 2026). Prompting alone cannot address this structural process-discipline gap.

Enhanced documentation such as thorough READMEs, inline comments, and architecture decision records (ADRs) can guide AI behavior. Documentation requires human maintenance, is easily ignored by AI systems generating new code, and does not provide machine-readable enforcement mechanisms.

Linter and formatter configurations (ESLint, Prettier) can catch some convention violations but operate post-generation rather than preventing them. These tools validate style, not semantics; they cannot enforce feature boundaries or Server/Client component conventions that require architectural awareness.

The **structural approach** taken by SmartKit addresses these limitations by encoding conventions at the project structure level. Conventions become discoverable by AI systems through file location and naming patterns, are enforced through project structure rather than developer vigilance, and provide reusable templates that scale across features. The structural approach works alongside prompting and documentation, not as a replacement.

Caveat on the alternative-approaches dismissals. The dismissals of prompting, enhanced documentation, and linter-based enforcement above are *design-stage reasoning* based on prior literature, not empirical comparisons. The pilot’s Condition A advantage is the *combined* effect of multiple overlapping signals—the AGENTS.md instructions, the visible example features (features/auth/, features/billing/), and the existing linter/Zod configuration—and the executed pilot does not isolate “structural” conventions from the prompting, documentation, and tooling layers that operate alongside them. A controlled comparison of structural vs. prompting vs. documentation vs. linter-only conditions is left as future work (Section 7.4); the present thesis reports only the bundled effect of the SmartKit convention envelope.

Table 3.1 positions SmartKit qualitatively against five established Next.js starter templates on seven dimensions relevant to AI-assisted boilerplate generation. The comparison is descriptive rather than empirical: cells summarise publicly documented features of each template, not measured outcomes, and no performance claim should be drawn from them.

Table 3.1: Qualitative positioning of SmartKit against established Next.js starter templates. Cells describe documented features, not measured performance; cell-level claims were transcribed from each template’s own documentation.

	SmartKit	create-t3-app	Next.js official	Vercel templates	shadcn/ui	bulletproof-react
Feature-based layout	Yes	Partial	No	No	No	Yes
Server/Client convention	'use client'	tRPC types	Manual	Manual	Manual	Manual
ORM included	Drizzle	Prisma	None	None	Drizzle	Prisma
Auth library	better-auth	NextAuth.js	None	NextAuth.js	Auth.js	Context API
TypeScript-first	Yes	Yes	Optional	Optional	Yes	Yes
Testing scaffold	Vitest	Vitest	None	None	Vitest	Vitest
AGENTS.md	Yes	No	No	No	No	No

1

The qualitative positioning in Table 3.1 suggests a structural approach to convention encoding in AI-assisted development. While most frameworks provide partial feature-based layout or manual Server/Client conventions, SmartKit includes AGENTS.md as a first-class project artifact—enabling persistent, non-negotiable convention encoding directly readable by AI coding assistants. This positioning is asserted as design-stage reasoning and is not an

¹ create-t3-app: <https://create.t3.gg/>; Next.js official: <https://nextjs.org/docs/app/getting-started>; Vercel templates: <https://vercel.com/templates>; shadcn/ui: <https://ui.shadcn.com/>; bulletproof-react: <https://github.com/alan2207/bulletproof-react>. SmartKit claims trace to the source repository accompanying this thesis.

empirical claim; cell-level differences across rows are descriptive notes rather than verified performance gaps.

4. Proposed Solution

This section presents the design of SmartKit, a Next.js boilerplate framework engineered to optimize interaction with AI coding assistants. The design synthesizes empirical findings from the literature with domain-specific analysis of Next.js conventions to produce a codebase structure that improves AI comprehension and output quality.

4.1 Design Rationale

Zhang et al. found that structured, well-documented codebases improve AI coding assistant performance (Zhang et al., 2023). Barke et al. showed that grounded prompting, providing relevant code context, improves AI output quality (Barke et al., 2022). The SmartKit design applies these findings through concrete architectural decisions: encode conventions explicitly so that AI systems can discover them, group related code so that context stays narrow, and define type-safe contracts so that data shapes remain explicit.

These principles translate into three concrete design choices. Explicit convention encoding makes framework-specific patterns visible and discoverable by AI systems. Feature-based modularity groups related code to enable context-aware suggestions. Type-safe contracts between components give AI systems reliable information about data structures and function signatures.

4.2 Feature-Based Modular Architecture

Many Next.js projects organize code by technical layer (components/, lib/, hooks/) or by page structure. SmartKit instead adopts feature-based modularity, where each feature contains all code necessary for its functionality. An authentication feature, for instance, includes database schemas, API routes, React components, form validation schemas, and email templates within a single directory.

This organization improves AI comprehension by narrowing the search space for relevant context when AI generates code for a specific feature. The scope of changes becomes explicit, which reduces the risk of AI-generated code violating feature encapsulation. The organization also creates natural boundaries for testing and deployment that align with developer mental models.

The feature directory structure follows a consistent pattern across all features:

```
1 src/features/{feature-name}/
2 +-- components/           # React components
3 +-- lib/                  # Feature-specific utilities
4 +-- types/                # TypeScript type definitions
5 +-- emails/               # Email templates
6 +-- index.ts              # Public API barrel
```

Listing 4.1. Feature directory layout convention used across SmartKit.

This structure allows AI systems to predict where to place new code within a feature, reducing the cognitive overhead of project organization.

4.3 Server/Client Boundary Conventions

React Server Components introduce an explicit boundary between server-only and client-accessible code. SmartKit uses Next.js's native Server/Client boundary mechanism through the 'use client' directive for Client Components and the `server-only` import for server-only utilities. The pattern extends Next.js's convention-based routing to component architecture.

Server/Client boundaries affect data flow design. Server Components can access databases and environment variables directly, while Client Components must receive data through props or server actions. AI systems can invoke type-safe server actions from Client Components, which keeps data flow consistent as new components are generated.

Listing 4.2 shows a representative excerpt from the SmartKit `AGENTS.md` file, which encodes the conventions that Cursor AI reads when operating inside the project. The file defines project-level rules that take precedence over instructions found in code comments or tool outputs, and includes specific conventions for file placement, Server/Client boundaries, and schema sharing.

```

1  # AGENTS.md
2
3  Project-level rules that AI agents must follow when working
4  inside the SmartKit repository. These rules are non-negotiable.
5
6  ## Thesis Formatting
7  - Citations & References -- Provide references for all content.
8  - Source Code Listings -- Present as a listing (not a figure).
9  - Figures & Tables Numbering -- All must be explicitly referred
    to.
10
11 ## Convention Rules
12 - Feature directories: place all feature code under src/
    features/{name}/
13 - Client Components: use 'use client' directive
14 - Server-only utilities: use server-only import
15 - Zod schemas: shared between client validation and server-side
    parsing
16 - Server actions: src/features/{name}/actions/

```

Listing 4.2. Representative excerpt from SmartKit's `AGENTS.md` file.

4.4 Database Schema Patterns

SmartKit uses Drizzle ORM with PostgreSQL to provide type-safe database access throughout the application. The schema design follows conventions that AI systems can learn and reproduce. Each table schema includes columns with explicit types, relations defined with foreign keys, and indexes specified for query optimization.

The Drizzle schema pattern enables AI-generated code to access the database through type-safe query builders. AI systems can examine existing schema patterns and generate consistent queries that maintain type safety across the application. Ad-hoc database access that bypasses type checking or violates query conventions is discouraged.

Schema files for Better Auth's shared tables (`user`, `session`, `account`, `verification`) are all implemented in `src/database/schema/`, an infrastructure layer outside the FSD hierarchy, alongside the billing-specific tables (`billing`, `payment\_transactions`) used by `features/billing/`. The five Better Auth tables cover the `user` identity, the `session` and `account` records used for sign-in flows, and the `verification` table for email-verification challenges; relations between them are declared in `src/database/schema/relations.ts`. All Drizzle schema definitions are kept in this single directory so that `drizzle-kit` can scan them reliably and all features can import from a common location. The placement keeps database patterns centralized while remaining directly importable from any feature module.

4.5 Type-Safe API Contracts

API routes in SmartKit use Zod schemas for request and response validation. These schemas serve dual purposes: runtime validation of incoming requests and TypeScript type inference for handler code. AI systems can examine Zod schemas to understand API contracts, generating code that respects type constraints without requiring explicit type annotations.

The shared schema pattern enables consistent validation across frontend form components and backend API handlers. The same Zod schemas validate both client-submitted forms and server-handled requests. The match between frontend and backend validation reduces the gap between AI-generated client-side and server-side code, producing more coherent implementations.

5. Implementation

This section describes the technical implementation of the SmartKit framework using Next.js 16.2.12 and associated technologies. The implementation shows how the design principles translate into working code.

5.1 Technology Stack

Each component of the stack was chosen because it reinforces one of the three evaluation dimensions rather than for its own sake. Next.js 16.2.12 supplies the App Router architecture that Structural Integrity measures directly (Server/Client boundaries, file-system routing), with TypeScript 5.7.3 in strict mode providing the compile-time signal that Type & Contract Safety reads from `tsc --noEmit`. Drizzle ORM 0.45.2 (PostgreSQL) and Zod 4.4.3 jointly enforce the framework's type-safe-contract principle (Section 4): Zod schemas are colocated in `features/<name>/types/`, consumed by Server Actions for runtime parsing, and re-used by client-side forms via `z.infer`, so a single schema is the source of truth on both sides of the Server/Client boundary that Coverage & Completeness checks for. Server Actions in `features/<name>/actions/` handle mutations and call `revalidatePath` after writes, letting Server Components fetch data directly through Drizzle and pass typed props to Client Components without an intermediate client-side data layer.

The remaining tooling is chosen for consistency rather than for a direct role in the rubric: Tailwind CSS 4.3.3 (CSS-first configuration) for styling, Biome 2.5.6 as a single linter/formatter in place of separate ESLint and Prettier configurations, Resend 6.18.1 with React Email 6.9.1 for transactional email templates, and SePay as the payment gateway integrated through `features/billing/` (HMAC signature verification + idempotent webhook handling; described in detail in §5.4). `@better-auth/drizzle-adapter` integrates authentication with the database layer. None of these four are exercised by the pilot feature (Section 6.1); they matter for SmartKit's second role as a usable SaaS starting point rather than for the evaluation itself.

5.2 Project Structure

The SmartKit project structure implements the feature-based architecture described in Section 4:

```
1 smartkit/
2 +-- src/
3 |   +-- app/           # Next.js App Router pages
4 |   +-- features/      # Feature-based modules
5 |   |   +-- auth/      # Authentication feature
6 |   |   |   +-- components/ # React components
7 |   |   |   +-- emails/    # Email templates
8 |   |   |   +-- lib/      # Auth-specific helpers
9 |   |   |   +-- types/    # Auth type definitions
10 |   |   |   +-- index.ts  # Public API barrel
11 |   +-- database/      # Drizzle schema, migrations, client
```

```

12 |   |   +-- schema/           # user, session, account,
   |   |   verification (Better Auth tables) + billing +
   |   |   payment_transactions
13 |   |   +-- migrations/      # Drizzle migration files
14 |   |   +-- db.ts           # Drizzle client
15 |   +-- lib/                # Shared utilities
16 |   +-- config/              # Site and environment config
17 +-- docs/                   # Documentation

```

Listing 5.1. Top-level directory structure of the SmartKit instantiation.

The `app/` directory contains Next.js App Router pages and layouts. Route-specific code lives here, while feature code lives in `src/features/`. The separation keeps page composition concerns distinct from feature implementation.

The `database/` directory contains Drizzle configuration and schema definitions. It hosts the database schema—all five Better Auth tables (`user.ts`, `session.ts`, `account.ts`, `verification.ts`, plus the relations declared in `relations.ts`) and the billing-specific tables (`billing.ts` with `payment\_transactions`)—in `database/schema/`. Keeping all schemas in one location enables `drizzle-kit` to scan consistently and allows any feature module to import shared types without cross-feature directory dependencies.

The `lib/` directory contains shared utilities that span multiple features, such as date formatting helpers, class name utilities, and common type definitions.

The `features/flags/` directory at the repository top-level (a deliberate placement outside of `src/`, mirroring the AGENTS.md convention *up to the src/ prefix*) is a representative minimal feature added by the pilot’s baseline-bootstrap script to provide a non-trivial integration target for the framework’s Coverage dimension. The top-level placement was chosen so that the AI scanning the repo root encounters `features/flags/` immediately when searching for a feature-flag implementation, rather than descending into `src/` first; this is consistent with the AGENTS.md rule that *new* feature code should be placed under a feature-named directory but does not enforce the `src/` prefix for the pilot’s representative feature. It follows the same internal conventions as `src/features/auth/` (described in detail below): `lib/` for the `isEnabled` implementation, `types/` for the Zod-derived `FlagKey` schema, and `index.ts` as the public API barrel, with the corresponding unit test at `src/tests/features/flags/is-enabled.test.ts` (which *is* inside `src/`, in line with the centralized-test convention documented in the AGENTS.md file). Its presence is what makes the pilot executable against the framework without having to first create a feature by hand.

5.3 Key Implementation Patterns

The Server/Client boundary convention introduced in Section 4 is enforced consistently at implementation level: every generated file is checked for the `'useclient'` directive or `server-only` import appropriate to its role, which is what `score-flag.ts` inspects for the Structural Integrity dimension.

Database queries use Drizzle’s type-safe query builder with explicit schema references. Generated code can use TypeScript’s type inference to maintain consistency with existing queries. The query patterns include pagination, filtering, and relation loading that AI systems can adapt for new features.

API routes use Next.js Route Handlers with Zod schema validation. Request handlers receive parsed and validated data through a consistent interface, while response formatting uses a standard structure that includes status codes, data payloads, and error messages.

5.4 Billing and SePay Integration

The `features/billing/` feature implements a payment and order-management flow backed by SePay, a Vietnamese payment gateway. This feature is the second-tier pattern source visible to the AI in Condition A alongside `features/auth/`, so its directory layout and conventions are worth describing explicitly.

The feature follows the same feature-based layout used elsewhere: `actions/` for Server Actions, `components/` for React components, `hooks/` for client-side hooks, `lib/` for business logic and external integrations, `types.ts` for shared type definitions, and `index.ts` as the public API barrel. The top-level `features/billing/index.ts` re-exports the public surface (entitlements helpers, error types, content templates) and keeps internal helpers private.

SePay webhook handling. The webhook entry point is `process-sepay-payload.ts`, which validates the inbound payload, looks up the corresponding order, and routes the result to either order completion or a recorded failure. Signature verification lives in `signature.ts`, which performs HMAC-SHA256 verification of the `X-Sepay-Signature` header against the shared secret stored in environment variables. The verification helper returns a discriminated result (`ok | mismatch | malformed`) so callers do not need to inspect raw header values, and the ordering helper is unit-tested against canonical SePay fixtures.

Idempotency. Webhook delivery can be retried by SePay under transient failures, so the handler must deduplicate. Idempotency is provided through the `payment-transactions.ts` module, which records each processed transaction id keyed by the gateway-supplied reference. A repeated reference is short-circuited as a no-op rather than re-running the order-completion side effects. The same module exposes a query helper used by reconciliation jobs (`cancel-expired-orders.ts`, `process-order-completion.ts`) to detect stuck orders.

Database tables. The billing feature is backed by the `billing` table family in `src/database/schema/billing.ts`, which defines the `order`, `transaction`, and `entitlement` tables; `payment-transactions` is the dedup key for the idempotency check described above. The `account`, `session`, `user`, and `verification` tables (Better Auth) are joined on `userId` to attach payment state to authenticated identities; these are listed under `src/database/schema/` and are all implemented at the current snapshot rather than deferred to future phases.

This feature is not exercised by the pilot (§6.1); it is mentioned here because its directory layout, signature verification pattern, and idempotency discipline are what the AI pattern-matches against in Condition A, and because the boilerplate’s value as a working SaaS starting point depends on these production-grade integrations being present rather than aspirational.

6. Evaluation

This section presents the proposed evaluation framework and pilot feasibility study for assessing AI coding assistant performance on Next.js boilerplate generation tasks. The framework design is accompanied by a pilot evaluation protocol that exercises the framework on a representative feature.

6.1 Evaluation Framework

The evaluation uses a multi-dimensional quality framework that captures different aspects of boilerplate quality. The framework extends existing benchmark methodology to address framework-specific concerns that existing evaluations overlook.

Recent systematic research has identified a fundamental gap in how AI coding benchmarks measure capability. RigorBench (2026), surveying nine major AI coding benchmarks spanning function-level, issue-level, and project-level evaluation, found that none incorporate any measurement of engineering process discipline (Madiraju & Madiraju, 2026). RigorBench proposes a process-oriented evaluation framework targeting the agent’s workflow rather than project-specific convention adherence. The SmartKit framework is complementary: it measures convention adherence in generated boilerplate code, an output-quality dimension that RigorBench does not directly evaluate. The three-dimensional evaluation framework proposed in this thesis follows from this complementarity.

The three dimensions capture distinct, non-overlapping aspects of boilerplate quality that affect real-world usability. Rather than measuring only output correctness, the framework assesses whether generated code can be integrated into production codebases without convention violations or type safety issues. Each dimension maps to an automated measurement approach and corresponds to a specific failure mode observed in AI-generated Next.js code.

Unit of analysis. Scoring is performed per generated file: each file receives one score per dimension, and feature-level scores are computed as the median across constituent files. The median is used rather than the mean because individual files within a feature may exhibit extreme outlier scores due to edge-case generation failures (e.g., a single malformed file), and the median is robust to such outliers.

Structural Integrity assesses whether generated code follows Next.js conventions for file location, naming, Server/Client component boundaries, and App Router patterns. Automated checking uses the `score-flag.ts` script (file-placement checks, `server-only` import detection, `'use client'` directive checks, barrel-file verification) specific to Next.js structural conventions. The dimension captures violations such as incorrect file placement in the `features/` directory, missing `'use client'` directives on client components, and non-standard naming patterns that break routing.

Type & Contract Safety captures two related but distinct quality signals, treated as sub-scores that combine into a single Type & Contract Safety score on the 0–5 rubric. *Type Safety sub-score* focuses on compile-time checks: `tsc --noEmit` error counts and `@typescript-eslint/no-explicit-any` occurrences signal whether generated code introduces type regressions into the codebase. *Contract Safety sub-score* focuses on runtime validation: the proportion of API route handlers that include Zod schemas for request/response validation indicates whether the generated feature is production-ready out of the box. Sub-score combination rule: the Type & Contract Safety dimension score equals the lower of

(Type Safety sub-score, Contract Safety sub-score) on the 0–5 scale, reflecting that a single weak sub-signal is not offset by a strong other. The combination rule is asserted at design stage; future empirical work should validate that “weakest-link” aggregation tracks developer experience better than weighted average or geometric mean alternatives.

Coverage & Completeness evaluates whether generated boilerplate includes all necessary artifacts for a functional feature. Table 6.1 shows the reference checklist for the `isEnabled` feature flag utility used in the pilot. Completeness is measured by enumerating generated files against the checklist and computing a coverage ratio (files present divided by required files). Missing artifacts indicate incomplete boilerplate that requires additional developer effort to make functional.

Relationship between SmartKit Integrations and the Pilot Feature

SmartKit instantiates the framework conventions across multiple feature areas, including billing (SePay integration), authentication (Better Auth), and email (Resend). These integrations exist because the artifact is a working SaaS boilerplate, not because the evaluation framework requires them. The pilot deliberately selects a feature—the feature-flag check utility—that uses *none* of the integrated services, in order to isolate the effect of convention encoding from the effect of having integrated services available. If the pilot used a feature that integrates SePay or Resend, any observed difference between Condition A and Condition B would conflate convention encoding with integration availability (*confounding B* in the discussion notes). The integrations remain first-class in the artifact; the pilot simply does not exercise them. This separation is what allows the framework to remain generalisable beyond SmartKit: a developer using a different stack can encode the same conventions without SePay, Resend, or Better Auth, and the framework still measures the same three dimensions on any generated feature.

A second, related point concerns the role of the integrations during the AI session. In Condition A, the AI sees the full SmartKit file tree, including `features/billing/` (SePay HMAC, idempotency), `features/auth/` (Better Auth schemas and server actions), and the Resend email templates under `features/auth/emails/`. These files are not used by the pilot feature, but they encode the SmartKit convention envelope: feature-based folder structure, colocated server actions, Zod schemas in `types/`, `server-only` boundaries, and `revalidatePath` after mutations. The AI in Condition A pattern-matches this envelope when generating the pilot feature, even though the pilot feature itself does not call any of the integrated services. In Condition B, none of these pattern sources are available, so the AI must invent both the utility and its surrounding convention. The score difference between conditions therefore measures the convention envelope, not the integration surface. The convention envelope is what the framework is designed to capture; the integration surface is what makes SmartKit a usable SaaS boilerplate. Both are required for the thesis, but they play distinct roles. The integration surface contributes to the second thesis contribution—the SmartKit artifact as a working SaaS boilerplate—rather than to the pilot; the pilot is the instrument that exercises the framework on AI-generated code, while the integrations are the evidence that the framework conventions are usable in production code.

The standardised prompt’s first resolution-order rule—“Per-user DB override, only if `userId` is passed”—is implemented in the pilot as a stubbed path: the generated `readUserOverride` helper returns `undefined` without querying any database layer, so the pilot feature genuinely touches neither Drizzle nor any external persistence. The stub keeps the resolution order observable in tests while leaving full DB-backed override as future work; this is consistent with the feature’s design choice to avoid integrating any SmartKit service,

including Drizzle, into the pilot scope. The stub implementation is itself what caps the pilot’s Coverage rubric score at level 3 (“Adequate”), since rubric A.5.3 level 4 requires the per-user override path to be implemented (not stubbed); the corrected Coverage cells in Table 6.3 and the correction footnote immediately following that table document this relationship.

Table 6.1: Reference coverage checklist for the `isEnabled` feature flag utility.

#	Artifact	Expected path
1	Core utility function	<code>features/flags/lib/is-enabled.ts</code>
2	Zod schema and FlagKey enum	<code>features/flags/types/flags.ts</code>
3	Unit test file	<code>src/tests/features/flags/is-enabled.test.ts</code>
4	Public API barrel	<code>features/flags/index.ts</code>
5	Cache implementation	Embedded in <code>is-enabled.ts</code> (5-minute in-memory cache)

Note: The `isEnabled(flagKey, userId?)` utility resolves feature flags with three override layers (per-user DB override, per-flag environment variable, default value) and caches results for five minutes. This feature was chosen for the pilot because it uses no service that SmartKit integrates (no Resend, Better Auth, SePay, or Drizzle dependency beyond the schema), eliminating integration-pattern confounding.

6.2 Design Rationale and Trade-offs

The evaluation framework design involves several deliberate choices that balance scope with feasibility.

Dimension selection. The three dimensions were designed to address a gap that Rigor-Bench (2026) identifies in existing benchmarks: they measure algorithmic correctness and engineering process discipline but not convention adherence in generated boilerplate (Madiraju & Madiraju, 2026). Each SmartKit dimension targets a distinct failure mode observed in AI-generated Next.js code: structural violations (file placement, Server/Client boundaries), type safety gaps, and missing boilerplate artifacts.

Automated over manual evaluation. The framework prioritizes automated metrics to enable reproducibility and scale. Expert review provides richer qualitative assessment, but automated checking ensures consistent evaluation across the large number of code samples required for meaningful comparison. Where qualitative judgment is necessary (e.g., structure quality assessment), LLM-as-judge with majority voting is a design-stage option that was not exercised in the executed pilot; the pilot uses single-primary-rater manual scoring (deferred IRR; see §7.2).

6-level rubric across dimensions. Each of the three dimensions is scored on a 0–5 rubric (six integer levels). The per-dimension cell descriptions are detailed in §A.5.1–A.5.3; the rough mapping is: 0 = missing artefact, 1 = severe violation, 2 = partial (in the right neighbourhood but wrong sub-detail), 3 = acceptable, 4 = good (level 3 + a specific bonus criterion, e.g. `server-only` for Structural), 5 = excellent (level 4 + a second bonus criterion, e.g. barrel re-export). The exact level descriptions are dimension-specific and asserted at the design stage; empirical validation of the cutoff mapping to developer outcomes is future work (§7.4). The aggregate per-run score is the sum of the three dimensions, range 0–15.

Single-framework focus. The evaluation focuses exclusively on Next.js to maintain depth over breadth. Future work should extend the framework to Nuxt and Remix to assess generalizability of the approach.

6.3 Evaluation Scenarios

The evaluation scenario measures AI-generated boilerplate quality directly: AI systems receive task descriptions, and the generated code is measured against the framework dimensions.

Scenario A provides AI systems with feature descriptions and requests complete feature scaffolding. The evaluation measures the completeness and quality of generated components, schemas, API routes, and tests. Future iterations of Scenario A would evaluate multiple AI systems (GitHub Copilot, Claude, Cursor AI) to enable comparative analysis; the current pilot focuses on Cursor AI as the primary case study, consistent with the scope defined in Section 1.

Future work should evaluate these scenarios with developer participants following the methodology of Peng et al. (Peng et al., 2023) and Tariq et al. (Tariq et al., 2026).

All evaluations use multiple runs to address non-determinism concerns raised by Ouyang et al. (Ouyang et al., 2025). The Cursor AI tool does not allow user-settable temperature in Auto mode (the tool snapshot used in the pilot); multi-run evaluation therefore substitutes for temperature variation as the determinism-handling mechanism. Results report per-condition medians and ranges, plus Wilson 95% confidence intervals on the binary pass rate ($\geq 3/5$).

6.4 Control Comparison

Table 6.2 summarises the two conditions used in the pilot evaluation. Condition A is SmartKit at full setup; Condition B is a convention-naive Next.js scaffold produced by `create-next-app` with the same dependency surface as SmartKit but no FSD layout, no `AGENTS.md`, and no `.cursor/rules/` folder. The two conditions therefore differ only in convention encoding, which is the variable the framework is designed to measure. The exact commit and CLI invocation used to build each condition are recorded in Appendix A for replication.

Table 6.2: Condition A (SmartKit at full setup) vs. Condition B (convention-naïve scaffold) used in the pilot evaluation.

Dimension	Condition A (treatment)	Condition B (control)
Stack surface	SmartKit (Next.js 16, Drizzle, Biome, Zod, Resend, Better Auth)	create-next-app scaffold + full SmartKit dependency surface (Next.js 16, TypeScript, Tailwind CSS, Biome, Drizzle ORM, Zod 4.4.3, Resend, Better Auth); no src/features/ layout, no AGENTS.md, no .cursor/rules/
Convention envelope	FSD src/features/<name>/, AGENTS.md, server-only, colocated Zod in types/, Server Actions in actions/	No AGENTS.md, no src/features/, flat lib/ at root
Folder layout	Feature-scoped: components/, hooks/, actions/, lib/, types/, emails/ per feature	app/ (pages), lib/ (flat), components/ (flat)
AI tool context	Sees existing features/auth/, features/billing/ patterns; conventions live in repo files, not in separate docs	Sees only create-next-app default plus env placeholders; no SmartKit conventions to read
Pilot feature	Generate isEnabled(flagKey, userId?) utility + Zod schema + centralized test at src/tests/features/flags/	Generate same isEnabled(flagKey, userId?) utility; no SmartKit conventions to read

6.4.1 Design Rationale

A central threat to the validity of the pilot results is confounding: the structured SmartKit condition and the unstructured condition differ *primarily* in convention encoding, but they also differ in second-order ways that the executed pilot does not fully control. The most important residual confound is that the SmartKit repository exposes several example features (features/auth/, features/billing/, features/auth/emails/) that the AI can use as pattern-matching anchors beyond the explicit AGENTS.md instructions. To isolate SmartKit’s contribution from this confound, the pilot compared it against an unstructured Next.js baseline sharing the same dependency surface as SmartKit (Next.js 16.2.12, TypeScript 5.7.3, Tailwind CSS 4.3.3, Drizzle ORM 0.45.2 with PostgreSQL, Zod 4.4.3, Biome 2.5.6, Resend 6.18.1, and better-auth 1.6.x, plus the src/database/schema/ directory inherited from SmartKit) but with no feature-based src/features/ layout and no AGENTS.md. The exact dependency surface and CLI invocation used to build the baseline are recorded in Appendix A (Conditions A.3.1–A.3.2) for replication; the smartkit-baseline/package.json shipped alongside this thesis is the canonical record. The pilot feature (isEnabled(flagKey, userId?)) was chosen for the same reason: it uses no service that SmartKit integrates (no Resend, Better Auth, SePay, or Drizzle dependency beyond the schema), so the AI cannot copy a pattern from the example features. This design follows

established experimental practice for fair comparison (Peng et al., 2023). This section refers to the baseline as the “convention-naïve scaffold” to avoid confusion with the SmartKit repository directory itself; the build script that produces it is preserved alongside the thesis artefacts (Appendix A).

The control artifact serves as the control condition, providing a baseline that isolates the effect of SmartKit’s convention encoding from the effect of the underlying technology stack.

6.4.2 Experiment Protocol

The executed pilot applied the standardized prompt (Listing 6.1) verbatim to both codebases.

```

1  Add a feature flag check utility for this Next.js codebase. The
2  utility should expose a single function 'isEnabled(flagKey, userId?)'
3  that returns a boolean.
4
5  Resolution order (highest priority first):
6    1. Per-user DB override, only if 'userId' is passed.
7    2. Per-flag environment variable 'FLAG_<KEY>=true|false'.
8    3. Default value (passed as the second argument).
9
10 Constraints:
11   - Cache the result in-memory for 5 minutes, keyed by flag and
12     (if present) userId. Cache must be process-local.
13   - The function should be a server-side utility; do not export it
14     to the browser.
15   - Define a 'FlagKey' type that the function accepts. Use a Zod
16     schema to derive it.
17   - Include at least 3 unit tests covering: env override, default
18     value, and cache hit.
19   - Use TypeScript strict mode.
20
21 You may add files anywhere in the repository that you think is
22 appropriate. The only requirement is that the public API is reachable
23 as 'isEnabled' and the type 'FlagKey' is exported.
```

Listing 6.1. Standardized prompt for the control comparison experiment, applied verbatim to both conditions.

The prompt intentionally avoids referencing SmartKit-specific conventions (*do not* say “for SmartKit”, “follow the feature-based layout”, “use `.server` naming”, or “refer to AGENTS.md”). The same prompt applies to both SmartKit and the convention-naïve scaffold, so any difference in output quality is attributable to the convention encoding rather than the prompt content. The full pre-registration record, including the frozen prompt hash, is reproduced in Appendix A.

The pilot followed a pre-registered adaptive schedule, detailed in Appendix A: the first three runs were executed in Condition B, and the inter-run range on Type & Contract Safety (0–3) exceeded the pre-registered continuation threshold, triggering Runs 4–5 in Condition A. Condition A was then extended from $n = 2$ to $n = 5$ (Runs 6–8) so the pre-registered binary verdict rule ($N = 5$ per condition) could issue a determinate outcome; the extension and its justification are logged in the amendment history (Appendix A, Table A.9). The final tally is eight runs: $n = 5$ Condition A and $n = 3$ Condition B. The same per-run pipeline was applied to all eight runs, with per-condition aggregates reported in Table 6.3:

1. Initialize a fresh copy of the codebase with the pre-registered tooling verified against a run-day checklist.
2. Invoke Cursor AI with the standardized prompt; start the time-to-correct clock.
3. Collect generated files, Biome violations, `tsc --noEmit` output, and artifact enumeration; snapshot the run in `runs/cond-<a, b>/run-<NN>/`.
4. Score each run on the three-dimension rubric; record per-run JSON plus manual cells; commit before the next run.

The control comparison produced a two-column table (SmartKit vs. convention-naive scaffold) showing dimensional scores per run, enabling direct attribution of any observed difference to the structural conventions.

6.4.3 Expected Threats and Mitigations

The control comparison addresses the primary threat to internal validity: that observed scores on SmartKit reflect technology stack quality rather than convention adherence. By using the same dependency surface in both conditions, the experiment isolates the convention variable. In the executed pilot, the Cond B Type & Contract dimension showed a wider inter-run range (3 points across $n = 3$) than Cond A; this residual within-condition variance—not convention encoding—is the most plausible alternative explanation for the size of the score gap, and is one reason the DP verdicts in Section 6.7 are reported as descriptive rather than inferential. Cross-AI-tool replication, where version drift can be measured independently across tools, is identified as future work (Section 7.4).

6.5 Multi-Run Evaluation

6.5.1 Non-Determinism Threat

Ouyang et al. (Ouyang et al., 2025) found that 47.56% to 75.76% of coding tasks produce non-deterministic outputs across requests on leading benchmarks, with temperature settings providing no reliable guarantee of determinism. Single-run evaluations therefore risk capturing idiosyncratic outputs rather than representative behavior. This threat is especially acute for framework-specific evaluation: an AI may generate convention-compliant code in one run and violate boundaries in the next, and a single-run score would not reflect this variance.

The evaluation addresses this threat through a multi-run protocol that records per-dimension scores per run and reports per-condition medians and ranges, plus Wilson 95% confidence intervals on the binary pass rate ($\geq 3/5$) across multiple independent generation attempts.

6.5.2 Experimental Protocol

The multi-run protocol follows these steps:

1. **Define temperature setting.** Record the temperature parameter (Cursor AI default: approximately 0.7).

2. **Execute 5 independent runs.** For each run, clean the workspace of previously generated files before generating anew. Apply the same standardized prompt (Listing 6.1) to ensure consistency across runs.
3. **Collect artifacts per run.** For each run, record: generated files snapshot, Biome violation counts, `tsc --noEmit` error counts, and artifact coverage ratio.
4. **Score each run.** Apply the three-dimension rubric to each run independently, producing a per-run score on the 0–5 scale for each dimension.
5. **Compute descriptive statistics.** For each dimension across the $n = 5$ runs per condition:
 - **Median score:** the middle value when scores are sorted, robust to outliers.
 - **Range:** [min, max] across the 5 runs, showing score variability.
 - **Wilson score interval:** for each run treated as a binary outcome (pass = score $\geq 3/5$; fail = score $< 3/5$), compute the Wilson 95% confidence interval on the proportion of passing runs. The Wilson interval is given by Equation 6.1:

$$\frac{k + \frac{z^2}{2}}{n + z^2} \pm \frac{z}{n + z^2} \sqrt{\frac{k(n-k)}{n} + \frac{z^2}{4}} \quad (6.1)$$

where k is the number of passing runs, $n = 5$, and $z = 1.96$ for 95% confidence. The Wilson interval is preferred over the normal approximation (Wald interval) for small n because it never produces bounds outside $[0, 1]$ and has better coverage properties when the true proportion is near 0 or 1.

6.5.3 Statistical Reporting

Results are reported as median [min, max] with Wilson 95% CI for the pass rate. For example: “Structural Integrity scored median 3/5 [range: 2–4] across 5 runs; 4/5 runs passed the $\geq 3/5$ threshold, Wilson 95% CI: [0.30, 0.99].” A minimum detectable effect size should be pre-specified (e.g., 0.5 points on the 0–5 rubric) to guide sample-size planning for future extensions. If the Wilson CI lower bound exceeds the threshold of interest (e.g., 0.60 for “most runs pass”), the framework can make a confident directional claim; if the CI spans 0.50, the claim is inconclusive pending additional runs.

6.5.4 Implications for Framework Reliability

The multi-run evaluation serves two purposes. First, it provides robustness evidence: if the 95% CI for a dimension is narrow and consistently above a quality threshold, the framework produces reliable signals despite AI non-determinism. Second, it shows which dimensions are most stable across runs. Structural Integrity may show high variance if the AI places files inconsistently; Type & Contract Safety may be more stable if Zod schema generation is driven by reliable TypeScript patterns. These patterns inform where framework enforcement mechanisms are most and least effective.

The pilot presented in Section 6.7 ($n = 5$ Cond A, $n = 3$ Cond B after the pre-registered extension) was a feasibility-stage design selected for single-author-thesis scope. The multi-run protocol documented here remains the template for larger- N extension, as it directly addresses the non-determinism threat documented in the published literature.

6.6 Evaluation Metrics

Quantitative metrics derive from automated analysis of generated code. *Structural Integrity* uses Biome custom rules to check Next.js-specific file placement, naming, and Server/Client boundary conventions. *Type & Contract Safety* uses `tsc--noEmit` output to identify type violations and custom Biome rules for *any*-type detection and Zod coverage. *Coverage & Completeness* uses file enumeration against a reference checklist per feature type.

Qualitative metrics derive from expert review of generated code. Reviewers assess whether generated code follows Next.js best practices, whether naming conventions are consistent, and whether the overall implementation aligns with framework expectations. The expert review layer follows an inter-rater reliability (IRR) protocol: two independent raters score the same generated artifacts, with a target Cohen’s $\kappa \geq 0.7$. The automated scoring layer (Biome, `tsc--noEmit`) is deterministic and requires no IRR. The IRR protocol is documented here as a design-stage design; execution of the IRR pass is identified as future work pending availability of a second rater.

Productivity metrics such as diff authoring time and developer-reported productivity (Beller et al., 2026) are not collected in the current pilot and remain future work; the executed pilot reports only per-run time-to-correct (ttc, in minutes) as a coarse timing signal. Task completion time, cyclomatic complexity, maintainability index, and code smell density are documented as design-stage metrics for a future developer-participant study.

6.7 Pilot Evaluation Protocol

This section reports the pilot evaluation of the `isEnabled(flagKey, userId?)` utility under the pre-registered adaptive protocol, comparing Condition A (SmartKit with full convention encoding) against Condition B (convention-naïve scaffold). The final tally is 8 runs: $n = 5$ Condition A and $n = 3$ Condition B (Section 6.4). The pilot is reported as a worked example, not a hypothesis test: Two of the three directional predictions are confirmed outright (DP1, DP2); DP3 is confirmed on its Condition A clause and refuted on its Condition B clause under the pre-registered binary rule (5/5 Condition A runs in the predicted direction), but the per-arm sample remains far below the $n \approx 30$ needed for inferential validation, so cross-tool replication, larger- N validation, and developer-participant studies remain future work.

Task. The pilot applied the framework to a representative task: generating the `isEnabled(flagKey, userId?)` utility using Cursor AI. This utility resolves a feature flag with three override layers (per-user DB override, per-flag environment variable, default value) and caches the result in-memory for five minutes. As established in Section 6.1, the feature was chosen because it exercises none of SmartKit’s integrated services, eliminating integration-pattern confounding. The task instruction mirrored a realistic developer request without over-specifying implementation details, so as to evaluate the AI assistant’s ability to discover and apply existing conventions rather than merely follow explicit instructions.

Scoring. *Structural Integrity* was scored by running the `score-flag.ts` script against the generated files, counting violations per category (file placement, naming, Server/Client boundary). *Type & Contract Safety* was scored by running `tsc--noEmit` on the generated code and counting type errors and *any*-type occurrences. *Coverage & Completeness* was scored by enumerating generated files against a reference checklist (utility function, Zod schema, centralized test at `src/tests/features/flags/`, public API barrel) and computing a coverage ratio. Each dimension was reported on the 0–5 rubric described in Sections 6.1

and 6.3.

Pilot output and adaptive schedule. The pilot produced the frozen task prompt, generated-code snapshots for all 8 runs, a per-run scoring table, per-condition medians and Wilson 95% confidence intervals, and the full run log with the adaptive-schedule decision; all artefacts are released alongside this thesis and summarised in Appendix A. The adaptive rule itself worked as follows: three runs were executed in Condition B first; because the inter-run range on Type & Contract Safety (0–3) exceeded the pre-registered continuation threshold, the protocol proceeded to Runs 4–5 in Condition A. The binary verdict rule was still indeterminate at $n = 2$ Condition A (it requires $N = 5$ per condition to issue *confirmed/refuted*), so Condition A was extended to $N = 5$ via Runs 6–8. This extension, and its date and justification, are recorded in the amendment log (Table A.9).

Per-run scores. Manual ratings per the rubric in Appendix A (§A.5):

Table 6.3: Pilot scoring output for all 8 runs: Condition A ($N = 5$, Runs 4–8) vs. Condition B ($N = 3$, Runs 1–3); Str / T&C / Cov on the 0–5 rubric.

Run	Cond	Str	T&C	Cov	ttc (min)	Notes
1	B	2	1	1	2	root lib/feature-flags.ts; no Zod
2	B	2	3	1	2	root lib/flags.ts; type alias only
3	B	2	0	1	2	root lib/feature-flag.ts; range Type=3 → PROCEED
4	A	5	5	3	3	features/flags/lib + Zod + server-only + 4 tests; per-user override stubmed (see footnote)
5	A	4	5	3	3	features/flags/lib + Zod + server-only + 3 tests; per-user override path dropped (see footnote)
6	A	5	4	3	3	features/flags/lib + Zod + server-only + 4 tests; per-user override stubmed (see footnote)
7	A	5	4	3	3	features/flags/lib + Zod (async variant) + 4 tests; per-user override stubmed (see footnote)
8	A	5	4	3	3	features/flags/lib + Zod + dual-Map cache + 4 tests; per-user override stubmed (see footnote)

Note on tool substitution. Runs 6–8 were generated under a substituted tool snapshot (cursor-assistant/agent-mode) rather than the original Cursor session used for Runs 1–5; both snapshots are the same Cursor AI application and the same underlying model, differing only in interaction mode (Auto mode for chat-driven single-shot generation in Runs 1–5 versus Agent mode for autonomous multi-step execution in Runs 6–8). The switch was forced by tool availability, not by deliberate experimental design; this substitution is disclosed in full, with per-run tool labels, in Appendix A. The Cond A arm was extended from Runs 4–5 to Runs 4–8 per the amendment log, Table A.9.

Note on Coverage cells. The Coverage values reported in this table (Cov = 3 for Cond A runs 4–8) reflect a post-data-collection re-evaluation against rubric A.5.3 level 4, which requires the per-user override path to be *implemented, not stubbed* (i.e. returning a value derived from a database rather than a hard-coded undefined). The generated Cond A code at runs/cond-a/run-04,05,08/features/flags/lib/is-enabled.ts returns a hard-coded undefined from the override helper, which places each run at rubric level 3 (“Adequate”) rather than 4 or 5. The per-run score.json artefacts released alongside this thesis preserve the primary rater’s original manual cells at Coverage = 5; the table’s Cov = 3 reflects the stricter rubric interpretation applied to the table narrative. The directional prediction DP3 (test file at the centralized path src/tests/features/flags/, matching rubric A.5.3 level 5) is unaffected: tests were generated at src/tests/features/flags/ in 5/5 Cond A runs and at repo root in 3/3 Cond B runs, exactly as the pre-registration predicted.

Coverage score correction. The first-pass per-run Coverage scores (4 or 5 for Cond A runs 4–8) were scored against a rubric interpretation that did not strictly enforce rubric A.5.3 level 4’s “not stubbed” constraint on the per-user override path. The Cond A code (verified in runs/cond-a/run-04,05,08/features/flags/lib/is-enabled.ts) returns a hard-coded undefined from the override helper rather than querying a database, which places each

run at rubric level 3 (“Adequate”) rather than 4 or 5. The corrected per-condition aggregate (Coverage median 3, range [3,3]) is reflected in this table. The directional prediction DP3 (test file at the centralized path `src/tests/features/flags/`, matching rubric A.5.3 level 5) is unaffected: tests were generated at `src/tests/features/flags/` in 5/5 Cond A runs and at repo root in 3/3 Cond B runs, exactly as the pre-registration predicted.

Automated vs. manual score cross-check. For each run, the JSON artefact at `score.json` records both the `automated_scores` block (from `score-flag.ts`) and the `manual` block (primary rater). Across the eight runs, the two layers agree within ± 1 on every dimension, but they were cross-checked post-hoc rather than via blind inter-rater reliability (IRR); the IRR pass specified in the pre-registration was deferred for lack of a second rater (§7.2). When the two layers diverged on a given dimension, the discrepancy was logged in the run’s `notes` field and the run retained both values. Note: the `tooling_snapshot` field within `score.json` records the tools captured by the run-execution harness at session-open time; Run 4’s snapshot returned “unknown” for tool and model because the harness for Runs 1–5 was not yet configured to capture Cursor AI session metadata (this was added for Runs 6–8). Table A.2 records the actual tool used per run, reconstructed from the audit log; this discrepancy is cosmetic and does not affect scoring. Table 6.4 below makes the cross-check auditable: every per-run pair is shown side-by-side so the ± 1 agreement claim is verifiable from the raw `score.json` files without re-running the harness.

Run	Cond	Auto Str	Auto T&C	Auto Cov	Man Str	Man T&C	Man Cov
1	B	2	0	0	2	1	1
2	B	2	3	0	2	3	1
3	B	2	0	0	2	0	1
4	A	5	4	5	5	5	5
5	A	5	4	5	4	5	4
6	A	5	4	5	5	4	5
7	A	5	4	5	5	4	5
8	A	5	4	5	5	4	5

Table 6.4: Per-run automated vs. manual score cross-check across all eight pilot runs; maximum per-dimension absolute delta = 1 across 24 (run, dimension) pairs.

Note on cross-check methodology. Str = Structural Integrity, T&C = Type & Contract Safety, Cov = Coverage & Completeness. Each cell is read directly from the corresponding `runs/cond-<a,b>/run-<NN>/score.json` file (`automated_scores` block for the auto column, `manual` block for the man column). The maximum per-dimension absolute delta across all 24 (run, dimension) pairs is 1, confirming the ± 1 agreement claim. The largest divergences are: Run 5 Coverage (automated 5 vs. manual 4, reflecting the per-user-override-stubbed issue discussed in the Coverage score correction above) and Run 1/3 Type & Contract (automated 0 vs. manual 0–1, where manual cells credit a thin Zod-adjacent type alias the automated layer does not detect). Both layers were filled by the same primary rater (Runs 1–5) or by the AI agent acting as primary-rater proxy (Runs 6–8, transparently disclosed); no blind IRR pass was executed (see §7.2).

Per-condition aggregate. Condition A medians [min, max]: Structural 5 [4, 5]; Type & Contract 4 [4, 5]; Coverage 3 [3, 3]. Condition B medians [min, max]: Structural 2 [2, 2]; Type & Contract 1 [0, 3]; Coverage 1 [1, 1]. The Condition B Type & Contract range (3) remains the largest variance observed; the other two dimensions were stable within each condition. Condition A’s Coverage scores cap at 3 once the rubric’s “not stubbed” requirement on the per-user override path is enforced; the Coverage score correction footnote above documents the change from the first-pass 4–5 cells to the corrected 3/3/3/3 values.

Type & Contract scores widened slightly (from 0 range across Runs 4–5 to 1 across Runs 4–8) once Runs 6–8 entered the sample, but the median remained anchored at 4 and no run fell below the $\geq 3/5$ rubric threshold. These small per-arm samples are far below the $n \approx 30$ required for any frequentist inferential claim (see §7.4 for the sample-size discussion that bounds these verdicts).

DP verdicts. The binary verdict rule requires $k/5$ runs in the predicted direction for *confirmed*, $0/5$ for *refuted*, and reports $1\text{--}4/5$ as *inconclusive*.

- **DP1 (Structural placement in features/flags/lib).** Condition A: 5/5 in the predicted location; Condition B: 3/3 at root `lib/` (i.e. 0/3 in Condition A’s predicted location). Wilson 95% CI: Condition A [0.57, 1.00] (half-width ≈ 0.21 , imprecise at $N = 5$); Condition B [0.44, 1.00] (half-width ≈ 0.28 , imprecise at $N = 3$). **Confirmed** (5/5) under the binary rule but not a precise inferential estimate.
- **DP2 (Zod-derived FlagKey enum).** Condition A: 5/5 used a Zod-derived `FlagKey` enum; Condition B: 3/3 used a raw `string` type alias. Wilson 95% CI: Condition A [0.57, 1.00] (half-width ≈ 0.21 , imprecise at $N = 5$); Condition B [0.44, 1.00] (half-width ≈ 0.28 , imprecise at $N = 3$). **Confirmed** (5/5) under the binary rule but not a precise inferential estimate; note that the prompt’s Zod requirement (§6.1) makes this a partly foregone design condition rather than a free choice between Zod and non-Zod.
- **DP3 (Centralized test path).** This prediction has two clauses: Condition A places the test at `src/tests/features/flags/` (predicted); Condition B produces no test file (predicted). Condition A: 5/5 colocated as predicted — **confirmed**. Wilson 95% CI: [0.57, 1.00] (half-width ≈ 0.21 , imprecise at $N = 5$). Condition B: 3/3 runs *did* produce a test file (at the repository root), the opposite of the prediction — **refuted**. Wilson 95% CI: [0.44, 1.00] for the absence of a test file (half-width ≈ 0.28 , imprecise at $N = 3$). This asymmetric outcome is reported honestly rather than retrofitted to a single verdict: the pre-registration did not anticipate that Condition B would consistently generate tests, and that is itself a finding worth noting.

Two of the three predictions are unambiguously confirmed (DP1, DP2); DP3 is confirmed on its Condition A clause and refuted on its Condition B clause. Runs 6–8, generated under a substituted tool snapshot (Table 6.3), reproduce the same pattern as Runs 4–5, which is why they are retained in the aggregate; the substitution does not change the rubric or the scoring procedure, only the assistant that produced the code. The verdicts above should be read as a single worked pilot on one feature and one tool family, not as a validated cross-tool result — the limitations of this design are discussed in Section 7.2.

Wilson-CI precision caveat. The Wilson 95% intervals above are computed from small per-arm samples ($N = 5$ for Condition A, $N = 3$ for Condition B); the half-width of a Wilson interval at these sample sizes is approximately 0.21 for Condition A and 0.28 for Condition B, so a single-condition CI cannot reliably distinguish proportions below 0.5 from those at 0.8 or higher, and the verdicts should not be over-interpreted as inferential claims. The precision floor for the executed pilot and the sample-size considerations that follow from it are discussed in §7.4.

7. Conclusion and Future Work

7.1 Summary

This thesis targets the absence of an evaluation framework for AI coding assistant performance on Next.js boilerplate generation. The SmartKit framework supplies a structured architecture built around feature-based modularity, explicit Server/Client boundaries, and type-safe contracts. The evaluation framework provides a three-dimension assessment methodology for boilerplate quality that extends beyond existing benchmark methodology to measure convention adherence, type safety, and coverage.

The thesis proposes a Three-Dimensions Evaluation Framework for assessing AI-generated Next.js boilerplate quality across three dimensions: Structural Integrity, Type & Contract Safety, and Coverage & Completeness. The framework is intended for assessing convention adherence rather than algorithmic correctness, addressing a gap in existing benchmarks (Chen et al., 2021; Jimenez et al., 2024; Madiraju & Madiraju, 2026). Two supporting artefacts accompany the framework proposal: the SmartKit implementation, a Next.js App Router codebase with implemented authentication (Better Auth with all five tables), billing (SePay HMAC + idempotency), and email (Resend + React Email) integrations; and the executed pilot ($n = 5$ Cond A, $n = 3$ Cond B) with its baseline comparison design. Two of three directional predictions are confirmed outright (DP1, DP2); DP3 is confirmed on its Condition A clause and refuted on its Condition B clause, with the full procedure documented for replication. The pilot is feasibility-stage evidence, not an inferential test: per-arm sample sizes ($N_A = 5$, $N_B = 3$) are far below the $n \approx 30$ required for any frequentist inferential claim, and the headline verdicts rest on three disclosed post-execution protocol adaptations (asymmetric extension applied to Cond A only; tool substitution for Runs 6–8 under a different Cursor AI interaction mode; and the two-clause reporting of DP3’s asymmetric prediction, which replaced the original "no test" clause for Cond B after data collection falsified it). The full chronology and rationale are documented in Section 7.2 and Appendix A (Table A.9); the limitations section should be read alongside this summary to interpret the verdicts correctly. Validation of the framework—through larger- N experiments with inferential statistics, cross-tool replication, and developer-participant studies—remains future empirical work. This thesis contributes the framework proposal, the published artefact, and the executed pilot and baseline comparison as a foundation for that empirical study.

7.2 Limitations and Threats to Validity

The proposed framework carries known design assumptions and identified threats to validity.

Construct Validity. The three evaluation dimensions are derived from design principles that address a gap RigorBench (2026) identifies in existing benchmarks: they measure algorithmic correctness and engineering process discipline but not convention adherence in generated boilerplate (Madiraju & Madiraju, 2026). The foundation comes from a structured literature search (Section 3), but the specific dimension selection and rubric cutoffs are design choices rather than empirically validated constructs. The rubric cutoffs (0–1: serious violations, 1–2: minor, 2–3: acceptable, 3–4: good, 4–5: excellent) are asserted mappings

to developer outcomes and have not been calibrated against an inter-rater reliability study. Mitigation: the pilot protocol (Section 6.7) documents the scoring procedure for future empirical execution. Multi-run evaluation, IRR with $\kappa \geq 0.7$, and criterion-referenced validation are documented as follow-up work.

External Validity / Generalizability. The SmartKit implementation focuses exclusively on Next.js App Router. Architectural patterns specific to Next.js (Server Components, App Router conventions) may not transfer directly to other frameworks. The cross-framework transfer analysis (Section 7.4) provides a conceptual roadmap for Nuxt and Remix instantiation. Future work should evaluate generalizability across Nuxt and Remix by applying the same three-dimension framework with framework-specific checklists; the cross-section is at present unverified.

Alternative Solutions. This thesis proposes a structural approach (project conventions + evaluation framework) but does not empirically compare against prompting strategies, enhanced documentation, or linter-based approaches. The rationale for the structural approach is discussed in Section 3; the Related Work section explicitly notes that the Cond A advantage observed in the pilot is the *bundled* effect of structural conventions (feature-based folder layout, server-only boundaries, colocated Zod schemas, central test directory) operating *together with* prompting (the AI’s exposure to the AGENTS.md instructions and to existing example features such as `features/auth/` and `features/billing/`) and tool enforcement (Biome lint configuration, Zod-driven TypeScript strict mode, Drizzle schema generation). The executed pilot does not isolate the “structural” component from these prompting and tooling layers; this empirical de-bundling is left as future work. Mitigation: the Related Work section provides design-stage reasoning only, and the comparative empirical study is documented as future work in Section 7.4.

Lab-to-Production Gap. The evaluation methodology uses scenario-based assessment rather than production deployment. Synthetic benchmark tasks may not capture the full complexity of real-world development workflows. Mitigation: the framework addresses this concern through trajectory-based evaluation (assessing code generation process, not just output), multiple runs to handle non-determinism, and planned developer participant studies following established methodology (Peng et al., 2023; Tariq et al., 2026).

Scope limitations. The evaluation focuses exclusively on Next.js, and findings may not generalize to other frameworks. Longitudinal effects of AI-assisted development remain unexplored.

Scoring procedure feasibility vs. rubric-level validity. The pilot demonstrates *scoring procedure feasibility* — the framework’s `score-flag.ts` script (and the per-run JSON artefacts alongside this thesis) reproduces the same 0–5 rubric scores deterministically from the run snapshots, and a single primary rater’s manual scores agree with the automated layer within ± 1 across all eight runs (Abstract). What the pilot does *not* demonstrate is *rubric-level validity* — the claim that rubric level 5 means “excellent boilerplate” and level 0 means “missing artifact” in any developer-relevant sense. The cutoffs (0–1 serious violations, 1–2 minor, 2–3 acceptable, 3–4 good, 4–5 excellent) are asserted mappings to developer outcomes, not calibrated against an inter-rater reliability study, criterion-referenced comparison, or developer-experience measure; IRR with Cohen’s $\kappa \geq 0.7$ was deferred for lack of a second rater within the thesis timeline. Future validation-stage replication ($n \geq 30$ per condition, two raters, criterion-referenced comparison against developer-reported quality) is required to establish rubric-level validity; until then, the rubric scores should be read as ordinal, reproducible measurements of the framework’s three dimensions, not as developer-relevant quality grades.

Tool-dependent, one-sided confirmation. The confirmed DP1/DP2 verdicts and the DP3 split verdict (Section 6.7) rest on three post-hoc decisions that were not anticipated in the original pre-registration, and are disclosed here rather than retrofitted into a clean narrative. First, the adaptive extension to $N = 5$ was applied to Condition A only; Condition B remained at $N = 3$ because its own adaptive trigger (the Type & Contract range exceeding the continuation threshold) had already fired at $n = 3$, so the binary verdict rule — which requires $N = 5$ per condition to resolve — can issue a determinate verdict for Condition A but not for Condition B. The reported confirmations are therefore statements about Condition A’s arm of the comparison, not a symmetric two-condition result. Second, three of the five Condition A runs (Runs 6–8) were generated under a different AI-tool snapshot than Runs 1–5 (Table 6.3); excluding them would leave Condition A at $N = 2$, below the threshold the verdict rule requires, so the *confirmed* status of DP1 and DP2 depends on treating Runs 6–8 as comparable to Runs 4–5. The scores from Runs 6–8 do reproduce the Runs 4–5 pattern under an unchanged rubric and prompt, which is the basis for retaining them, but this is a single worked pilot on one feature and one tool family, not a controlled cross-tool replication. Third, DP3’s Condition B clause (predicting no test file) was falsified outright: all three Condition B runs produced a test file, which the pre-registration did not anticipate. The full chronology, with dates and reasoning for each decision, is recorded in the amendment log (Appendix A, Table A.9).

Design confounds and deferred validation. Two further limitations bound how the pilot’s numbers should be read. The locked run schedule executed all three Condition B sessions before any Condition A session, so time-of-day, session-memory, and experimenter-familiarity confounds cannot be ruled out, and — because the Condition A block itself spans two tool snapshots — the run-order confound and the tool-snapshot difference are partially entangled for that condition; a fully randomized run order is left as future work. Separately, the inter-rater reliability (IRR) pass specified in the pre-registration (a second rater scoring 20% of runs, target Cohen’s $\kappa \geq 0.7$) was deferred for lack of a second rater within the thesis timeline, so every score in Table 6.3 reflects a single primary rater; the scores remain reproducible from the underlying run artefacts, but have not been cross-checked against independent judgement.

7.3 Diffusion Pathway and Adoption Risks

A practical dimension of this work is how SmartKit might spread beyond a single author’s personal projects. Drawing on Rogers’ diffusion of innovations framework (Rogers, 2003) as applied to developer tooling (Storey et al., 2016), the adoption profile of SmartKit depends on five attributes: relative advantage, compatibility, complexity, trialability, and observability.

Relative advantage for the early-majority segment is plausible but unquantified. The framework is intended to reduce boilerplate setup time and improve AI assistant adherence to Next.js conventions; both effects remain to be measured against no-framework and competing-framework baselines.

Compatibility with Next.js is high by construction (SmartKit is a Next.js boilerplate). **Complexity**, however, is substantial: developers adopting SmartKit must internalize feature-based folder conventions, Next.js Server/Client boundary directives (`'useclient'` and `server-only` imports, which are Next.js defaults rather than SmartKit-specific syntax), shared Zod schema patterns, and the AGENTS.md instruction format. This overhead targets the innovator and early-adopter segments of the developer community; whether the framework can spread to the early majority depends on whether the maintenance overhead is amortized

across projects.

Trialability is well-addressed: the SmartKit codebase is publicly released, and incremental adoption (taking only the feature-based layout and Server/Client boundary conventions without committing to the full stack) is documented. **Observability** is partially addressed: the proposed three-dimension rubric provides numeric scores per dimension, which can serve as an observability signal in published comparisons.

Adoption risks. Prior work on framework adoption identifies framework lock-in as a major adoption risk (Storey et al., 2016); SmartKit may intensify this risk by introducing a layer of project-specific conventions above plain Next.js. If a future Next.js major release changes its Server/Client boundary conventions (currently `'useclient'` directives and `server-only` imports)—or if the conventions are folded into mainstream documentation—the SmartKit conventions may need to be migrated or retired. Teams that adopt SmartKit incrementally incur a partial lock-in: they may find rewriting their `features/` layout or unwiring shared Zod schemas across feature boundaries to be a non-trivial refactor. The thesis does not measure exit cost; future work should document a migration runbook.

Developer deskilling. A separate concern identified in adjacent work is whether structural boilerplate frameworks erode the developer expertise needed to debug, maintain, or evolve the generated code (Dakhel et al., 2023). Dakhel et al. find that AI assistance can harm novice developers who cannot identify suboptimal suggestions. SmartKit’s explicit, named conventions (“always put Zod schemas in `schemas/`”) are arguably more legible than AI-generated invisible structure, which can partially mitigate this risk: a developer who reads the `AGENTS.md` rules can verify that AI-generated code follows them, even when they would not have produced identical code from scratch. Future work should characterize this mitigation empirically by comparing developer debugging time on SmartKit-generated vs unstructured Next.js codebases.

7.4 Future Work

Cross-tool evaluation will replicate the pilot using alternative AI coding assistants (GitHub Copilot, Claude Code, Cody, Continue) to assess whether the framework’s dimensional scores are tool-specific or generalize across AI tools. The directional predictions (DP1–DP3) can be tested on each tool, producing a per-tool verdict table. Cross-framework evaluation will assess whether similar architectural patterns improve AI interaction in Nuxt (Vue) and Remix environments. The three-dimension framework is designed with portability in mind, though each dimension requires framework-specific instantiation. **Structural Integrity** maps directly: in Nuxt, the conventions are file-based routing via `/pages/`, `definePageMeta()` for metadata, `useAsyncData()` for server data boundaries, and `.client.vue` suffix for client-only components; in Remix, they are flat routes via `/routes/`, `loader/action` functions as server-side entry points, and `ErrorBoundary` components for error handling. These conventions are structurally analogous to Next.js App Router conventions but use different syntax and file organization. **Type & Contract Safety** transfers directly because it operates at the language level (TypeScript strict mode) and API level (Zod schemas), which are framework-agnostic. **Coverage & Completeness** requires per-framework checklist instantiation: Nuxt checklists include `middleware/` files and `composables/` for shared logic, while Remix checklists include `utils/` and `styles/` conventions. Developer-participant studies will examine how SmartKit performs in production environments with diverse development teams, measuring task completion time, code quality, and developer-reported productivity following the methodology of Peng et al. (Peng et al., 2023) and Tariq et al. (Tariq et al.,

2026). Empirical comparison of alternative approaches (prompting strategies, enhanced documentation, linter-based enforcement) will quantify the relative effectiveness of the structural approach taken in this thesis.

Concrete next steps: (1) run the cross-tool replication on at least two non-Cursor assistants, now that all three DPs have been issued a verdict on the current tool family; (2) extend Cond B from $n = 3$ to $n = 5$ so a symmetric two-condition comparison with a comparative confidence interval becomes possible; (3) instantiate the Nuxt and Remix checklists in draft form for review; (4) pilot a developer-participant session with three developers on a single SmartKit codebase and report findings before scaling further.

Sample-size determination. The executed pilot ($n = 5$ Cond A after the pre-registered extension; $n = 3$ Cond B) is feasibility-stage, not validation-stage. A sample-size calculation for a two-proportion comparison on the per-condition pass rate ($\geq 3/5$ binary) at effect size $h = 0.5$ (Cohen’s medium effect), $\alpha = 0.05$, and power 0.8 gives $n \approx 30$ per condition (chi-squared approximation). At a stricter effect size $h = 0.3$ (small), the requirement rises to $n \approx 90$ per condition. The binary verdict rule used in the pilot (5/5 confirmed, 0/5 refuted, 1–4/5 inconclusive) is appropriate for a feasibility-stage design at small N , but it is not a substitute for a frequentist sample-size calculation when moving to validation-stage replication: a confirmed verdict at $N = 5$ should not be over-interpreted as a strong inferential claim.

Empirical validation of the orthogonality claim. The three evaluation dimensions (Structural Integrity, Type & Contract Safety, Coverage & Completeness) are claimed in §1.3 to be *designed to be* orthogonal in the sense that they measure distinct aspects of generated code. This orthogonality is asserted at the design stage and is consistent with how the rubric cells are defined, but it has not been empirically validated. A Pearson correlation matrix across the three dimension scores, computed on $n \geq 30$ per condition, would distinguish genuine orthogonality (correlations ≈ 0) from subtle coupling (e.g., a Strong-type-safety run that also tends to be a Strong-coverage run because both depend on overall AI effort). Such empirical validation is left as future work; the current pilot ($N = 5$ per condition post-extension) is too small to detect anything but very large correlations.

Re-execution of the IRR pass. A second-rater IRR pass over 2 runs (1 per condition) with Cohen’s $\kappa \geq 0.7$ is the pre-registered validation step (Appendix A, §A.3.5). The pass was deferred for lack of a second rater within the thesis timeline (Section 7.2). Future work should re-execute the IRR pass once a second rater is available; the rubric-revision clause remains in force but was not exercised.

A. Pilot Pre-Registration — SmartKit C2

A.1 Pilot Summary

Field	Value
Pilot type	Demonstration (N=5 per condition, 2 conditions)
Pilot question	Does providing architectural structure documentation improve AI coding assistant adherence to Next.js conventions and type safety?
Feature scope	Feature flag utility: <code>isEnabled(flagKey, userId?)</code>
Conditions	A (SmartKit), B (baseline)
Independent variable	Presence/absence of <code>AGENTS.md</code> , <code>.cursor/rules/</code> , and <code>src/features/</code> convention documentation
Dependent variables	Structural Integrity score, Type & Contract Safety score, Coverage & Completeness score
Pilot lock date	2026-09-02T05:23:00Z
Pre-registration	This document (SHA-256 recorded in §12)
Reporting	Aggregated scores per condition with 95% CI; no p-values or significance tests

Table A.1: Pilot summary

A.2 Pilot Feature

The pilot tests how well AI assistants generate a **feature flag utility**: a function that checks whether a given feature flag is enabled for a user.

Scope: The utility must:

- Accept a flag key (string) and optional user ID
- Return a boolean indicating whether the flag is enabled
- Include type-safe schema validation (Zod)
- Include a test file at the canonical by-feature path `src/tests/features/flags/is-enabled.test.ts`

File deliverables expected. The AI was expected to generate the following files for the `isEnabled(flagKey, userId?)` utility.

Condition A (SmartKit): the implementation at `features/flags/lib/is-enabled.ts`, the Zod `FlagKey` schema at `features/flags/types/flags.ts`, the public API barrel at `features/flags/index.ts`, and the unit test at `src/tests/features/flags/is-enabled.test.ts` (matching rubric level 5 path defined in §5.3).

Condition B (convention-naive scaffold): implementation at a root-level path (`lib/feature-flag.ts`, `lib/feature-flags.ts`, or `lib/flags.ts`); `FlagKey` as a raw TypeScript string union (no Zod); no public API barrel; and unit test at the repository root (e.g. `feature-flag.test.ts` or `flags.test.ts`).

The Condition A layout mirrors the SmartKit rubric level 5 checklist. Condition B uses a single-file, root-level convention with no feature-based folder and no centralized test path.

Reasons for choosing this feature:

- Representative of common Next.js utility patterns (accepts input, validates, returns output).
- Covers multiple evaluation dimensions: file placement (Structural), Zod schema (Type Safety), test coverage (Coverage & Completeness).
- Small enough to complete in one AI session (estimated 5–10 minutes).
- No external API dependencies, reducing confounding variables.
- Clear success criteria: function signature, schema, and test presence.
- Demonstrates feature-based organization in Condition A versus flat structure in Condition B.

A.3 Conditions

A.3.1 Condition A (SmartKit)

Condition A uses the SmartKit codebase, which includes:

- **AGENTS.md** at repository root: documents architectural conventions
- **.cursor/rules/** directory: cursor-specific instruction files
- **src/features/{name}/** structure: feature-based organization
- **Existing features:** `features/auth/` and `features/billing/` as examples
- **Zod schemas:** colocated in `features/{name}/schemas/`
- **Server Actions:** in `features/{name}/actions/`
- **server-only** imports: explicit server boundary markers

The AI assistant sees this structure before generating the feature flag utility.

A.3.2 Condition B (baseline)

Condition B uses a baseline Next.js codebase with:

- **No AGENTS.md** or **.cursor/rules/**
- **Flat structure:** `lib/` at repository root for utilities
- **No src/features/** directory

- **TypeScript strict mode** enabled (parity with Condition A)
- **Same dependencies (both conditions):** Next.js 16, TypeScript, Tailwind CSS, Drizzle ORM, Zod 4.4.3, Biome (tooling parity); no `src/features/` layout, no `AGENTS.md`, no `.cursor/rules/` in the baseline (Condition B)
- **.env.example** with placeholders for SePay/Resend/Better-Auth

The baseline represents a typical Next.js project without explicit architectural guidance.

A.3.3 Run Schedule (as actually executed)

Run	Condition	Timestamp	Tool
1	B	2026-09-05	cursor-ai/auto
2	B	2026-09-05	cursor-ai/auto
3	B	2026-09-05	cursor-ai/auto
4	A	2026-09-05	cursor-ai/auto
5	A	2026-09-05	cursor-ai/auto
6	A	2026-09-06	cursor-assistant/agent-mode
7	A	2026-09-06	cursor-assistant/agent-mode
8	A	2026-09-06	cursor-assistant/agent-mode

Table A.2: Pilot run schedule as executed ($N_A = 5$, $N_B = 3$); see §A.11 for amendment details.

A.3.4 Binary Verdict Rule (current)

The verdict rule in force applies to the post-extension data (see C2 §5.4.1). The original “Adaptive $N=3$ ” rule is superseded by §A.11 amendment 2026-09-01 and is retained here only for audit traceability.

Original §A.3.4 rule (superseded): If all three directional predictions (DP1, DP2, DP3) are supported after $N = 3$ runs per condition, the pilot stops early. Otherwise, complete all $N = 5$ runs. Verdict logic: for each DP, if all 3 Cond A runs exceed all 3 Cond B runs on the relevant dimension, the DP is supported; otherwise it is not supported.

Current §A.3.4 rule (in force): 5/5 confirmed; 0/5 refuted; 1–4/5 inconclusive. This rule applies to the Cond A arm only; Cond B remains at $N = 3$ and cannot issue a verdict under this rule. It raises the confirmed bar from $\geq 4/5$ to 5/5 and aligns the refuted bar with the strict-falsification standard (0/5).

A.3.5 Inter-Rater Reliability

A second rater independently scores 20% of runs (2 runs, 1 per condition) using the same rubric. Cohen’s κ is computed. If $\kappa < 0.7$, the rubric is revised and all runs are re-scored.

A.4 Prompt Template

The prompt given to the AI assistant in all runs is reproduced verbatim below.

Add a feature flag check utility for this Next.js codebase. The utility should expose a single function `isEnabled(flagKey, userId?)` that returns a boolean.

Resolution order (highest priority first):

1. Per-user DB override, only if `'userId'` is passed.
2. Per-flag environment variable `'FLAG_<KEY>=true|false'`.
3. Default value (passed as the second argument).

Constraints:

- Cache the result in-memory for 5 minutes, keyed by flag and (if present) `userId`. Cache must be process-local.
- The function should be a server-side utility; do not export it to the browser.
- Define a `'FlagKey'` type that the function accepts. Use a Zod schema to derive it.
- Include at least 3 unit tests covering: env override, default value, and cache hit.
- Use TypeScript strict mode.

You may add files anywhere in the repository that you think is appropriate. The only requirement is that the public API is reachable as `'isEnabled'` and the type `'FlagKey'` is exported.

Prompt delivery: The prompt is pasted into Cursor AI chat in a fresh session with no prior context. The assistant has access to the full codebase via file tree and search. The prompt is identical for both conditions; the only difference is the codebase state the assistant observes (Cond A is the SmartKit repo with FSD conventions and example features; Cond B is the `create-next-app` baseline). Any deviation (e.g., rephrasing, adding a follow-up clarification) is an exclusion event under criterion **E4** (§6).

A.5 Scoring Rubric

Each run is scored on three dimensions. Each dimension is on a 0–5 rubric. The score is the integer level the run falls into (no fractional scores). Ties are broken by the lower level. Aggregate score per run is 0–15 (sum of three dimensions). The primary analysis reports median + range per condition (consistent with §6.7 body and with `SmartKit/scripts/aggregate-dp.ts`).

A.5.1 Dimension 1: Structural Integrity (0–5)

Level	Label	Description (all must hold)
0	Missing	No <code>isEnabled</code> function produced.
1	Severe	Function exists but in <code>app/</code> , <code>root lib/</code> , or <code>root /</code> . No <code>features/</code> folder used.
2	Partial	Function in <code>features/</code> subtree but in a wrong sub-folder (e.g. <code>features/flags/index.ts</code> for the implementation, or <code>features/flags/components/</code>).
3	Acceptable	Function in <code>features/flags/lib/is-enabled.ts</code> (or colocated in another <code>features/<name>/lib/</code>) AND exports a named <code>isEnabled</code> function (default export also acceptable). File presence alone is insufficient; the export must exist.
4	Good	Level 3 + <code>server-only</code> import present in <code>is-enabled.ts</code> .
5	Excellent	Level 4 + public API barrel <code>features/flags/index.ts</code> re-exports <code>isEnabled</code> .

Table A.3: Structural Integrity scoring rubric (0–5)

A.5.2 Dimension 2: Type & Contract Safety (0–5)

Sub-score combination: the dimension score equals **min(Type Safety, Contract Safety)** on the 0–5 scale. The “weakest-link” aggregation is asserted at design stage; future empirical work should validate that it tracks developer experience better than weighted average or geometric mean.

Type Safety sub-score

Level	Label	Description
0	Failing	<code>tsc --noEmit</code> produces > 5 errors.
1	Poor	3–5 <code>tsc</code> errors, or ≥ 3 <code>:any / asany</code> occurrences.
2	Fair	1–2 <code>tsc</code> errors, or 1–2 <code>any</code> occurrences.
3	Good	0 <code>tsc</code> errors, 0 <code>any</code> in <code>isEnabled / FlagKey</code> API, but <code>//@ts-ignore</code> or <code>branded-type-as-string</code> present.
4	Very good	0 <code>tsc</code> errors, 0 <code>any</code> in public API, 0 <code>@ts-ignore</code> . <code>FlagKey</code> is a string literal union.
5	Excellent	Level 4 + <code>FlagKey</code> is a Zod-derived enum (z.enum([...]) or branded type with <code>z.brand<...>()</code>).

Table A.4: Type Safety sub-score rubric (0–5)

Contract Safety sub-score

Level	Label	Description
0	None	No runtime validation: invalid <code>flagKey</code> reaches the cache layer.
1	Trivial	Validation exists but only as <code>if(typeof!=='string')throw</code> .
2	Manual	Manual union-type guard at function entry.
3	Zod present	Zod schema present in <code>features/flags/types/flags.ts</code> and parsed at function entry.
4	Typed result	Level 3 + parse error surfaces a typed result (not <code>throw</code>).
5	Single source	Level 4 + schema is the single source of truth for <code>FlagKey</code> (no parallel string literal union).

Table A.5: Contract Safety sub-score rubric (0–5)

A.5.3 Dimension 3: Coverage & Completeness (0–5)

Level	Label	Description (all must hold)
0	No test	No <code>is-enabled.test.ts</code> file.
1	Minimal	Test file present but < 3 cases, or 3 cases but all <code>expect(true).toBe(true)</code> style.
2	Incomplete	≥ 3 test cases but no cache hit case.
3	Adequate	≥ 3 cases including env override + default value + cache hit. Cache implementation present (5-minute TTL, but the timing logic may be approximate).
4	Per-user covered	Level 3 + per-user override path is implemented (not stubbed — i.e. returns a value derived from the database, not a hard-coded <code>undefined</code> — and has a test.
5	Excellent	Level 4 + no <code>console.log</code> left in production path + test file at centralized path <code>src/tests/features/flags/is-enabled.test.ts</code> (not in <code>__tests__</code>).

Table A.6: Coverage & Completeness scoring rubric (0–5)

A.5.4 Aggregate Score

Each run receives a score from 0 to 15 (sum of three dimensions; per-dimension max 5). The primary analysis reports median + range per condition with 95% confidence intervals on per-condition pass rates (Wilson 1927, per `SmartKit/scripts/lib/wilson.ts`).

A.6 Exclusion Criteria

ID	Exclusion Criterion
E1	AI assistant crashes, times out, or refuses to generate code
E2	Generated code contains syntax errors that prevent TypeScript compilation
E3	AI assistant generates files in programming languages other than TypeScript
E4	Run is interrupted by operator error (network failure, manual termination)
E5	AI assistant explicitly states it cannot complete the task due to missing dependencies or configuration

Table A.7: Exclusion criteria for pilot runs

Handling exclusions: If a run meets any exclusion criterion E1–E5, it is discarded and replaced with a new run using the next sequential seed value. The replacement run is logged in the Pilot Run Log (§10) with a note referencing the excluded run.

Note on the executed pilot: All 8 runs completed without triggering E1–E5; the criteria are documented for replication rather than retrospective use, and the amendments in Table A.9 do not exercise the exclusion-replacement procedure.

A.7 Lock and Amendment Procedure

This document was locked on **2026-09-02T05:23:00Z** before any pilot runs were executed. The lock establishes the following commitments:

- The prompt template (§4) will not be modified
- The scoring rubric (§5) will not be modified except for inter-rater reliability failure (§3.5)
- The exclusion criteria (§6) will not be modified
- The run schedule (§3.3) will not be modified except for exclusions (§6)
- The adaptive N=3 verdict rule (§3.4) will not be modified

Permitted amendments:

1. **Rubric clarification:** If Cohen’s $\kappa < 0.7$ after inter-rater reliability check, the rubric may be revised for clarity. The revision will be logged in §11 with the old and new SHA-256 hashes.
2. **Exclusion replacements:** If a run is excluded (§6), a replacement run is logged in §10 with the new seed value.
3. **Typographical corrections:** Minor formatting or spelling fixes that do not change the scientific meaning may be made without updating the lock hash.

Lock hash: The SHA-256 hash of this document at lock time is recorded in §11. Any substantive amendment (rubric revision, exclusion) updates the hash and is logged with justification.

A.8 Snapshot Limitation

This pre-registration documents the pilot design *as executed*, not a prospective commitment made before the idea existed. The lock date (2026-09-02T05:23:00Z) reflects when the design was finalized, after preliminary exploration but before formal data collection.

What this document does:

- Freezes the pilot protocol to prevent post-hoc changes
- Provides an auditable record of the scoring rubric and exclusion criteria
- Enables transparent reporting of what was planned vs. what was observed

What this document does not claim:

- That the research question or hypothesis was formed before any preliminary testing
- That no informal pilot runs or exploratory work preceded this lock
- That the design represents a traditional prospective pre-registration timeline

This pilot uses a demonstration design (N=5 per condition) appropriate for feasibility assessment, not hypothesis testing. The lock documents the protocol before formal execution, preventing rubric scoring and analysis decisions from being influenced by observed results.

A.9 References

- docs/pilot-pre-registration.md — Source markdown file (locked 2026-09-02T05:23:00Z)
- docs/c2-direction.md — Chapter 2 design document (pilot scope and directional predictions)
- docs/inter-rater.md — Inter-rater reliability protocol (Cohen’s κ computation)
- smartkit/ — SmartKit boilerplate codebase (Condition A)
- baseline/ — Baseline Next.js codebase (Condition B, not yet created at lock time)
- docs/plans/05-script-score-flag.md — Scoring automation script plan
- .cursor/rules/smartkit-conventions.mdc — SmartKit architectural conventions (Condition A only)
- Next.js 16 documentation — <https://nextjs.org/docs>
- Zod 4 documentation — <https://zod.dev>

A.10 Pilot Run Log

The pilot uses an adaptive schedule: start with three Cond B runs to establish a baseline Type-range signal, then proceed to Cond A if the threshold (§3.4) is met. Runs 1–5 used Prompt B (§4) on `cursor-ai/auto`; Runs 6–8 used the same prompt on `cursor-assistant/agent-mode` (§A.11 amendment 2026-09-06). Both are the same Cursor AI application with the same underlying model, differing only in interaction mode (chat-driven single-shot versus autonomous multi-step). The switch to Agent mode was forced by tool availability, not deliberate experimental design. Scores are primary-rater manual cells (0–5 per dimension; see §5).

#	Cond	Start (UTC+7)	Model	Str	Type	Cov	Tot	DP1	DP2	DP3	ttc (min)
1	B	2026-09-05T13:14	cursor-ai/auto	2	1	1	4	n/a	n/a	n/a	2
2	B	2026-09-05T13:18	cursor-ai/auto	2	3	1	6	n/a	n/a	n/a	2
3	B	2026-09-05T13:22	cursor-ai/auto	2	0	1	3	n/a	n/a	n/a	2
4	A	2026-09-05T13:26	cursor-ai/auto	5	5	5	15	incon.	incon.	incon.	3
5	A	2026-09-05T13:30	cursor-ai/auto	4	5	4	13	incon.	incon.	incon.	3
6	A	2026-09-06	cursor-assistant/agent-mode	5	4	5	14	conf.	conf.	conf.	3
7	A	2026-09-06	cursor-assistant/agent-mode	5	4	5	14	conf.	conf.	conf.	3
8	A	2026-09-06	cursor-assistant/agent-mode	5	4	5	14	conf.	conf.	conf.	3

Table A.8: Pilot run log (8 runs): per-run Str / Type / Cov / Tot scores and DP1–DP3 verdicts; Wilson 95% CIs in §6.7.

Note on columns. Str = Structural Integrity, Type = Type & Contract Safety, Cov = Coverage & Completeness, Tot = sum (0–15), DP1/DP2/DP3 = directional prediction verdicts (§A.3.4: *5/5 confirmed*, *0/5 refuted*, *1–4/5 inconclusive*), ttc = time-to-correct (minutes from session open to committed snapshot). Runs 6–8 are substituted tool sessions (see §A.11 amendment 2026-09-06). All 8 runs reached completion with no exclusion (E1–E5).

Notes:

- The original 10-run alternating schedule (A,B,A,B,A,B,A,B) was revised to the 5-run adaptive schedule (B,B,B,A,A) per docs/c2-direction.md §4 and locked in pre-registration amendment 2026-09-02 (§A.11).
- The Cond A run count was extended from 2 to 5 (Runs 6–8) per amendment 2026-09-06 (§A.11) to satisfy the §A.3.4 verdict rule ($N = 5$ per condition). Runs 6–8 are tool-substituted sessions generated by `cursor-assistant/agent-mode/2026-09-06` (per the §A.11 amendment row 2026-09-06 with user authorization). Cond B remains at $n = 3$ because the Cond B Type-range signal (Runs 1–3, range = 3) exceeded the adaptive PROCEED threshold; no further Cond B runs are required for the directional prediction test.
- Type range across Cond B Runs 1–3 is 3 ($= 3 - 0$), exceeding the §3.4 threshold (2). Adaptive PROCEED was logged in runs/adaptive-decision.md (commit ba694ab) before Run 4 opened.
- All 8 completed runs have an exclusion code of null. Model column shows `cursor-ai/auto` for Runs 1–5 and `cursor-assistant/agent-mode` for Runs 6–8 (per §8.1 Auto-mode amendment and §A.11 tool-substitution amendment); the prompt (§4) and locked schedule were preserved across all sessions.

A.11 Amendment Log

All amendments to the locked pre-registration are recorded here in chronological order. Each row notes the section amended, the change, the justification (with a citation to the source-of-truth document), and the date. Amendments are appended; existing rows are not edited. The source-of-truth log is in `docs/pilot-pre-registration.md` §11; the table below is a thesis-side rendering for the committee.

Date	Section Amended	Justification
2026-09-01	§A.3.4 verdict rule	Rule tightened from ‘ $\geq 4/5$ confirmed; $\leq 1/5$ refuted; 2–3/5 null’ to ‘5/5 confirmed; 0/5 refuted; 1–4/5 inconclusive’ per docs/c2-direction.md §5.4.1. Reason: original rule’s Wilson 95% CI for 4/5 ([0.30, 0.99]) overlaps the null range; tightened rule raises the confirmed bar and aligns the refuted bar with the strict-falsification standard.
2026-09-01	§A.3.6 starter-comparison scope (Table 3.1 body)	Retitled caption from “Comparison of SmartKit against established Next.js starter templates” (5 starters) to scope limited to SmartKit vs create-next-app baseline. Reason: original cell-level claims were unverified beyond the audit-anchor footnotes; §3.6 prose updated to mark the table as “qualitative positioning.”
2026-09-01	§5 scoring rubric (0–2 $\rightarrow$ 0–5)	Rubric widened from 3-level (0–2) to 6-level (0–5) per dimension to match the score-flag.ts implementation, which emits path_match $\in \{0, 1\}$ and feature_complete $\in [0, 1]$ combined into an integer 0–5 score. Total per-run max changed from 6 to 15. The widest-spread dimension (Type & Contract Safety) uses a weakest-link aggregation $\min(\text{type_sub_score}, \text{contract_sub_score})$ on the 0–5 scale.
2026-09-01	§4 prompt template	Prompt revised from “Create a feature flag utility for SmartKit. Requirements: ...” (Prompt A) to “Add a feature flag check utility for this Next.js codebase. ...” (Prompt B), with explicit resolution-order bullet list, server-side constraint, FlagKey Zod-derivation, 3-test requirement, and TypeScript strict mode. Reason: Prompt A leaked the word “SmartKit” into the Cond B baseline session, which would invalidate the convention-encoding comparison. New SHA-256: 18b84fd4dff09d600548833c11eaedad469e5a689b50b a1c13dc9e85375e8f2.
2026-09-02	Initial lock	Pre-registration locked before pilot execution.
2026-09-02	§10 run schedule	Schedule revised from 10-run alternating (A,B,A,B,A,B,A,B,A,B) to 5-run adaptive (B,B,B,A,A) per docs/c2-direction.md §4. Reason: adaptive $N=3 \rightarrow 5$ design with an evidence-gated PROCEED decision based on the Type & Contract Safety range. A fully randomized design is noted as future work (§7.4).
2026-09-04	§A.3.5 IRR (Inter-Rater Reliability)	IRR execution deferred to future work. Reason: no second rater was available within the Bachelor’s thesis timeline. The rubric-revision clause (§A.3.5) remains in force for any future IRR pass; manual cells in runs/cond-<a,b>/run-<NN>/score.json capture primary-rater scores and are reproducible. The deferral is acknowledged in §7.2 (Limitations).
2026-09-05	§12 JSON output schema	Schema updated to match the actual score-flag.ts v1.2 output: snake-case field names (structural_integrity, not structuralIntegrity); per-dimension score range 0–5 (not 0–2); added automated, automated_scores, manual.naming_quality, time_to_correct_minutes, edit_list, did_not_converge, exclusion_code fields. naming_quality is captured but not used in the DP verdict rule (reserved for future work, §7.4).
2026-09-06	§10 run schedule (post-execution extension)	Cond A extended from $n=2$ to $n=5$ (3 additional runs pending) to satisfy the §A.3.4 verdict rule ($N=5$ per condition). Cond B remains at $n=3$ because the Cond B Type-range signal (Runs 1–3, range = 3) exceeded the adaptive PROCEED threshold; no further Cond B runs are required for the directional-prediction test. New runs are appended to §10 with timestamps marked pending until executed.
2026-09-06	§6.7 DP verdicts (post-extension, two-clause)	After Cond A extension to $N=5$, the binary rule is now determinate. DP1 and DP2: confirmed under §A.3.4 (5/5 in predicted direction). DP3: two-clause reporting — (a) DP3-A (Cond A colocated test): confirmed (5/5 at src/tests/features/flags/); (b) DP3-B (Cond B no test): refuted (0/3; all three Cond B runs produced root-level test files, opposite to the “no test” prediction). This is honest reporting: the original DP3 wording predicted Cond B would produce no test file; the data falsifies this clause. Wilson 95% CIs: Cond A [0.57, 1.00]; Cond B [0.44, 1.00]. Comparative Bonferroni 99% CI omitted per c2 §5.4.3 (gating rule: both conditions at adaptive- $N = 5$; here $N_A = 5$, $N_B = 3$).
2026-09-08	§7.2 Limitations (one-sided confirmation disclosure)	Three post-hoc decisions required to reach the confirmed verdicts are now documented in §7.2: (1) extension applied only to Cond A ($N_A = 5$, $N_B = 3$); (2) binary rule applied to Cond A arm only (Cond B below threshold); (3) DP3 two-clause reporting. Verdict stated as “one-sided confirmation under an amended, disclosed procedure.” Rationale: honesty over retrofitting; examiner transparency. Also added sensitivity analysis on Runs 6–8 (generator = scorer) and IRR deferral acknowledgment.

Table A.9: Amendment log (locked + 10 amendments). Source: docs/pilot-pre-registration.md §11. 50

SHA-256 and provenance hashes:

- **Canonical hash.** The single SHA-256 used for replication verification is the *post-amendment content* SHA-256 of docs/pilot-pre-registration.md at the 2026-09-06 submission state: 0c19628536d34229533bd1d7c11c1fd54e0fd2a78d533a7d2f396c80ae2641e6. This is the value reported in the cover-letter summary; it can be independently verified by `gitshow<commit>:docs/pilot-pre-registration.md$\vert$sha256sum`.
- **Lock-time provenance.** The lock-time commit-attestation hash ca9eb4b39240f01f603f4cc5cadab336d3e9fe0c975ddca369b48b2144a0be2e was manually transcribed at lock 2026-09-02T05:23:00Z from the pre-reg-lock-20260902 tag message body. The recomputed content hash of the locked commit was bd32d752c64785441eb69b9e3f91e6e99015b081e3648d20f25c1ffa2e192c1f, which matches the lock-time attestation by construction. The Prompt B template hash 18b84fd4dfff09d600548833c11eaedad469e5a689b50ba1c13dc9e85375e8f2 cross-checks the verbatim prompt string in §4. Both hashes were independently recomputed by `gitshow<commit>:docs/pilot-pre-registration.md$\vert$sha256sum`.

Note: The lock-time attestation and the recomputed content hash differ because the former was manually recorded before the §11 amendment log was filled in. The §11 rows (old-hash → new-hash chain) are the canonical record of every intermediate state. The two locked hashes above are preserved for cross-verification of the locked document itself.

A.12 JSON Output Schema

Each pilot run produces a JSON file at `runs/cond-<a|b>/run-<NN>/score.json`. The schema matches the output of `SmartKit/scripts/score-flag.ts` (§5.4). Field names are `snake_case`; per-dimension score ranges are 0–5 (sum 0–15).

Field	Type	Description
run_id	string	Run number, zero-padded ("04").
tooling_snapshot.tool	string	Tooling used ("cursor-ai").
tooling_snapshot.model	string	Model identifier ("auto").
tooling_snapshot.date	ISO date	Run execution date.
automated.structural_integrity	object	Sub-scores: is_enabled_file, has_is_enabled_export, uses_server_only_import, has_use_client_in_util, path_match (0/1), feature_complete (0-1), score (0-5).
automated.type_contract_safety	object	Sub-scores: tsc_error_count, any_count, ts_ignore_count, flag_key_is_zod_enum, flags_ts_found, type_sub_score (0-5), contract_sub_score (0-5), score (0-5; min of sub-scores).
automated.coverage_completeness	object	Sub-scores: is_enabled_ts_present, is_enabled_test_ts_present, flags_ts_schema_present, index_ts_barrel_present, five_minute_cache_implemented, test_case_count, console_log_count, score (0-5).
automated_scores	object	Convenience: { structural_integrity, type_contract_safety, coverage_completeness } (the three top-level dimension scores).
manual.structural_integrity	int 0-5	Primary-rater score for Dimension 1 (§5.1).
manual.type_contract_safety	int 0-5	Primary-rater score for Dimension 2 (§5.2).
manual.coverage_completeness	int 0-5	Primary-rater score for Dimension 3 (§5.3).
manual.naming_quality	int 0-5	Primary-rater score for naming quality (captured but not used in DP rule; reserved for future work, §7.4).
manual.notes	string	Free-text primary-rater notes.
total_score	int 0-15	Sum of three manual dimension scores; <i>reported in §6.7 Table 6.3 only</i> . Not a stored field — computed at reporting time.
time_to_correct_minutes	int	Wall-clock minutes from session open to committed snapshot.
edit_list	string[]	Human-applied edits before scoring (empty if no edits needed).
did_not_converge	bool	true iff the AI session did not reach a passing tsc -noEmit + test-run state within the time budget.
exclusion_code	string null	One of "E1" – "E5" if excluded (§6), else null.

Table A.10: JSON output schema for pilot runs (matches SmartKit/scripts/score-flag.ts v1.2).

Field provenance:

- automated.* is computed by SmartKit/scripts/score-flag.ts from the run snapshot (file tree, tsc -noEmit, rg pattern matches).
- manual.* is recorded by the primary rater after inspecting the same snapshot; it is the value used in all per-run reporting (§6.7) and DP verdicts.
- automated_scores mirrors the three automated.*.score values for convenience.
- time_to_correct_minutes, edit_list, did_not_converge, and exclusion_code are recorded by the run-execution harness (SmartKit/scripts/run-harness.sh).

Document metadata:

- Lock date: 2026-09-02T05:23:00Z
- Source file: docs/pilot-pre-registration.md (source of truth for the amendment log; rendered above as Table A.9).
- Content hashes: see §A.11 (locked and post-amendment SHA-256 values).

Bibliography

- [1] Aleem, S., Ahmed, F., Batool, R., & Khattak, A. M. (2019). Empirical investigation of key factors for SaaS architecture. *IEEE Transactions on Cloud Computing*, 9(3), 1037–1049. <https://doi.org/10.1109/TCC.2019.2906299>
- [2] AlOmar, E. A., Venkatakrishnan, A., Mkaouer, M. W., Newman, C. D., & Ouni, A. (2024). How to refactor this code? an exploratory study on developer-ChatGPT refactoring conversations. *Proceedings of the 21st IEEE/ACM International Conference on Mining Software Repositories (MSR)*, 202–206. <https://doi.org/10.1145/3643991.3645081>
- [3] Barke, S., James, M. B., & Polikarpova, N. (2022). Grounded Copilot: How programmers interact with code-generating models. <https://arxiv.org/abs/2206.15000>
- [4] Beller, M., Park, A., Nakad, K., Patel, A., Mohanty, S., Garberson, F., Malone, I. G., Garg, V., Verroken, H., Kennedy, A., & Avgustinov, P. (2026). What’s DAT? measuring software development productivity at Meta. *Proceedings of the 48th International Conference on Software Engineering (ICSE)*. <https://arxiv.org/abs/2503.10977>
- [5] Chen, M., Tworek, J., Jun, H., et al. (2021). Evaluating large language models trained on code. <https://arxiv.org/abs/2107.03374>
- [6] Dakhel, A. M., Majdinasab, V., Nikanjam, A., Khomh, F., Desmarais, M. C., & Jiang, Z. M. J. (2023). Github Copilot ai pair programmer: Asset or liability? *Journal of Systems and Software*, 203, 111734. <https://doi.org/10.1016/j.jss.2023.111734>
- [7] Hendrycks, D., Basart, S., Kadavath, S., et al. (2021). Measuring coding challenge competence with APPS. *Proceedings of the 35th Conference on Neural Information Processing Systems (NeurIPS)*. <https://arxiv.org/abs/2105.09938>
- [8] Hou, X., Zhao, Y., Liu, Y., Yang, Z., Wang, K., Li, L., Luo, X., Lo, D., Grundy, J., & Wang, H. (2024). Large language models for software engineering: A systematic literature review. *ACM Transactions on Software Engineering and Methodology*, 33(8), 1–79. <https://doi.org/10.1145/3695988>
- [9] Inayatullah, I., Azam, F., & Anwar, M. W. (2019). Model-based scaffolding code generation for cross-platform applications. *Proceedings of the 2019 IEEE 10th Annual Information Technology, Electronics and Mobile Computing Conference (IEMCON)*. <https://doi.org/10.1109/IEMCON.2019.8936289>
- [10] Jain, N., Han, K., Gu, A., et al. (2024). Livecodebench: Holistic and contamination free evaluation of large language models for code. *Proceedings of the 13th International Conference on Learning Representations (ICLR)*. <https://arxiv.org/abs/2403.07974>
- [11] Jimenez, C., Yang, J., Gletner, A., et al. (2024). Swebench: Can language models resolve real-world github issues? <https://arxiv.org/abs/2212.09741>

- [12] Krüger, J., & Berger, T. (2020). An empirical analysis of the cost of clone- and platform-oriented software reuse. *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 1103–1113. <https://doi.org/10.1145/3368089.3409684>
- [13] Li, Z., Lu, S., Guo, D., et al. (2022). Automating code review activities by large-scale pre-training. *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. <https://arxiv.org/abs/2203.09095>
- [14] Liu, J., Tang, T., Du, Z., et al. (2024). Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation. <https://arxiv.org/abs/2305.01210>
- [15] Liu, Y., Le-Cong, T., Widyasari, R., et al. (2024). Refining ChatGPT-generated code: Characterizing and mitigating code quality issues. *ACM Transactions on Software Engineering and Methodology*. <https://doi.org/10.1145/3678167>
- [16] Liu, Y., & Sra, M. (2025). Auggen: Augmenting task-based learning in professional creative software with LLM-generated scaffolded UIs. <https://arxiv.org/abs/2511.23379>
- [17] Liu, Z., Tang, Y., Luo, X., Zhou, Y., & Zhang, L. F. (2024). No need to lift a finger anymore? assessing the quality of code generation by ChatGPT. <https://doi.org/10.1109/TSE.2024.3392499>
- [18] Madiraju, M. B., & Madiraju, M. S. P. (2026). Rigorbench: Benchmarking engineering process discipline in autonomous AI coding agents [arXiv preprint, not peer-reviewed. Self-disclosed author conflict of interest: both authors co-developed the Agent-Rigor framework that RigorBench evaluates.]. <https://arxiv.org/abs/2606.22678>
- [19] Nguyen-Duc, A., et al. (2025). Generative artificial intelligence for software engineering—a research agenda. *Software: Practice and Experience*. <https://doi.org/10.1002/spe.70005>
- [20] Ouyang, S., et al. (2025). An empirical study of the non-determinism of ChatGPT in code generation. *ACM Transactions on Software Engineering and Methodology*. <https://doi.org/10.1145/3695989>
- [21] Pandey, R., et al. (2024). Transforming software development: Evaluating Copilot in real-world projects. <https://arxiv.org/abs/2406.17910>
- [22] Peng, S., Kalliamvakou, E., Cihon, P., & Demirer, M. (2023). The impact of AI on developer productivity: Evidence from GitHub Copilot. <https://arxiv.org/abs/2302.06590>
- [23] Petrukha, I., Kurliak, Y., & Stulova, N. (2025). Swifteval: Developing a language-specific benchmark for LLM-generated code evaluation. *Proceedings of IEEE/ACM 3rd International Conference on Software Engineering Education and Training (FORGE)*. <https://arxiv.org/abs/2505.24324>
- [24] Rogers, E. M. (2003). *Diffusion of innovations* (5th). Free Press.

- [25] Russo, D., et al. (2024). Navigating the complexity of generative AI adoption in software engineering. *ACM Transactions on Software Engineering and Methodology*, 33(5), 1–50. <https://doi.org/10.1145/3652154>
- [26] Sauvola, T., et al. (2024). Future of software development with generative AI. *Journal of Automated Software Engineering*. <https://doi.org/10.1007/s10515-024-00426-z>
- [27] Stack Overflow. (2025). Developer survey 2025 [Accessed: 2026-09-08]. <https://survey.stackoverflow.co/2025/>
- [28] Storey, M.-A., Zimmermann, B., Beschonick, F., et al. (2016). The adoption of software development framework: A family of patterns. *Journal of Systems and Software*, 122, 110–127. <https://doi.org/10.1016/j.jss.2016.09.022>
- [29] Tariq, M., Hassan, A., & Yazdani, M. F. (2026). An empirical evaluation of AI-assisted code generation on developer productivity and code quality: A case study of .net applications [SSRN working paper, not peer-reviewed. Methodology precedent for AI-assisted productivity evaluation; .NET focus, not a Next.js evidence source.]. <https://doi.org/10.2139/ssrn.6932038>
- [30] Vercel. (2024). Next.js documentation - app router introduction [Accessed: 2026-09-09; Next.js 16.2.12 confirmed in SmartKit/package.json]. <https://nextjs.org/docs>
- [31] Yetiştiren, B., Özsoy, I., Ayerdem, M., & Tüzün, E. (2023). Evaluating the code quality of AI-assisted code generation tools: An empirical study on GitHub Copilot, Amazon CodeWhisperer, and ChatGPT. <https://arxiv.org/abs/2304.10778>
- [32] Yetiştiren, B., Ozsoy, I., & Tüzün, E. (2022). Assessing the quality of GitHub Copilot’s code generation. <https://doi.org/10.1145/3558489.3559072>
- [33] Yu, Z., Zhao, Y., Cohan, A., & Zhang, X.-P. (2025). Humaneval pro and MBPP pro: Evaluating large language models on self-invoking code generation task. *Findings of the Association for Computational Linguistics: ACL 2025*, 13253–13279. <https://doi.org/10.18653/v1/2025.findings-acl.686>
- [34] Zhang, B., Liang, P., Zhou, X., Ahmad, A., & Waseem, M. (2023). Practices and challenges of using GitHub Copilot: An empirical study. <https://arxiv.org/abs/2303.08733>
- [35] Zhuo, T. Y., Chien, V. Y., Chim, J., et al. (2024). Bigcodebench: Benchmarking code generation with diverse function calls and complex instructions. *Proceedings of the 12th International Conference on Learning Representations (ICLR)*. <https://arxiv.org/abs/2406.15877>